AI-Driven Company-Wide Search Engine
for Deloitte Resources

Final Report

Group Two | ITWS 4100 Capstone | April 9, 2026

Professor: Richard Plotka

Raven Levitt Project Manager	**Felix Tian** System Architect	**Matthew Voynovich** Security Researcher
Sophia Turnbow User Researcher	**Jesse Gabriel** Developer	**Andrew Jung** Market Researcher

Section 1: Executive Summary

Deloitte is the largest professional services network in the world, employing over 470,000 consultants, auditors, tax advisors, and specialists across more than 150 countries. These professionals depend on a vast repository of internal resources, including documents, policies, slide decks, training materials, and past deliverables, to serve their clients effectively. However, searching for specific resources across Deloitte's fragmented internal platforms is often difficult and time-consuming, particularly when employees cannot remember the exact title, author, or location of what they need.

To address this challenge, Deloitte partnered with a team of six Information Technology & Web Science and Computer Science students from Rensselaer Polytechnic Institute: Jesse Gabriel, Andrew Jung, Raven Levitt, Felix Tian, Sophia Turnbow, and Matthew Voynovich. Together, the team designed and developed a working prototype of an AI-powered search engine that enables Deloitte employees to find internal resources using natural language queries. Rather than requiring users to know exact keywords or navigate multiple platforms, the search engine interprets user intent through Natural Language Processing and returns relevant results ranked by a hybrid retrieval pipeline combining semantic understanding, keyword matching, and neural reranking.

The project delivered four primary deliverables as specified in Deloitte's project brief: defined use cases and requirements grounded in user research, market research on NLP security and query boundaries, technical architecture documentation with diagrams addressing business logic and security, and a functioning web application prototype. The technology stack is entirely open-source-first, built on Next.js, FastAPI, PostgreSQL with pgvector and ParadeDB, and Qwen3-Embedding-0.6B, eliminating the per-user licensing fees that commercial alternatives would impose at Deloitte's scale.

A cost-benefit analysis was conducted to evaluate the project's financial viability by quantifying productivity recovery and identifying key risks. Using a 10% discount rate and conservative assumptions (70% adoption among a 50-person pilot cohort, 1.5 hours per week recovered per adopter), the analysis determined a net present value of +$2,168,378 over three years, an internal rate of return of approximately 3,700%, and a payback period within the first month of operation. The total risk exposure was assessed at $16,600 in expected monetary value, well within manageable bounds. Every financial scenario tested, including worst case, base case, and best case, yielded a positive NPV.

The project followed a nine-week Agile development timeline, prioritizing regular client feedback and weekly milestones. The team utilized Jira, specifically Kanban boards and Calendar view, to manage tasks and monitor progress. The team also coordinated through GitHub for version control, Google Drive for document collaboration, and Discord for daily communication. Key team members met weekly with Deloitte stakeholders to provide updates, gather feedback, and ensure alignment on the project's goals. The prototype was demonstrated with a live demo, showcasing hybrid search across a curated corpus of 80+ sample documents in PDF, DOCX, and PPTX formats.

This project represents a proof-of-concept that demonstrates what becomes possible when a search engine understands both user intent and the complex structure of an organization's internal resources. The architecture is designed with a clear path to production: enterprise authentication integration, multilingual support, expanded corpus ingestion, and evolution from document retrieval to retrieval-augmented answer generation.

Table of Contents

AI-Driven Company-Wide Search Engine 1
for Deloitte Resources 1
Section 1: Executive Summary 2
Table of Contents 4
Section 2: Introduction 5
Section 3: Client Organization & Description 5
Section 4: Project Team 6
Section 5: Problem Statement 8
Section 6: Strategy & Outline for the Solution (Mini-Strategic Plan) 9
6.1 Industry Definition and Market Size 9
6.2 Five Forces Analysis 9
6.3 Competitive Landscape 10
6.4 Internal Resources and Capabilities 10
6.5 VRIN Analysis 11
6.6 Business-Level Strategy: Focused Differentiation 11
6.7 Build vs. Buy Rationale 12
6.8 Strategic Risks 12
Section 7: Analysis Methodology 13
7.1 Development Methodology 13
7.2 User Research 13
7.3 Market Research 17
7.4 Financial Analysis Approach 18
Section 8: IS/IT Design (Requirements, Code, Screenshots, Database) 19
8.1 System Requirements 19
8.2 Technical Architecture 21
8.3 Search Pipeline Design 22
8.4 Document Ingestion Pipeline 24
8.5 Database Design 25
8.6 Security Architecture 25
8.7 Key Design Decisions 26
8.8 Frontend and User Experience 27
8.9 Deployment Architecture 28

8.10 Working System and Screenshots....29
Section 9: Financial & Cost-Benefit Analysis.... 29
9.1 Cost Analysis....29
9.2 Benefit Analysis....30
Tangible Benefits....30
Intangible Benefits (Qualitative)....31
9.3 Financial Metrics (ROI, NPV, IRR, Payback)....32
Return on Investment (ROI)....32
Net Present Value (NPV)....32
Internal Rate of Return (IRR)....32
Payback Period....33
9.4 Sensitivity Analysis....34
Break-Even Analysis....34
9.5 Risk Assessment....35
Section 10: Project Management Plan & Schedule.... 37
10.1 Team Organization and Roles....37
10.2 Development Timeline....37
10.3 Communication and Collaboration Tools....39
10.4 Challenges and Lessons Learned....40
Section 11: Post-Turnover Plan including Education, Training, Maintenance, and Support.... 41
11.1 Documentation and Knowledge Transfer....41
11.2 Training and Onboarding....41
End User Orientation (Analysts and Managers)....41
Administrator Training (Admin Role)....42
IT Operations Training....42
11.3 Maintenance and Ongoing Support....42
Routine Maintenance (Monthly)....42
Periodic Updates (Quarterly)....42
Support Plan....42
11.4 Recommended Production Roadmap....43
Section 12: Results....43
Section 13: Summary and Conclusions.... 43
References.... 44

Section 2: Introduction

Deloitte employees rely on a variety of internal sites to access company documents, policies, slide decks, and images. However, searching for specific resources across these platforms can be difficult, especially when users are not sure how to describe what they need or must navigate multiple tools. Deloitte identified this as a problem worth solving and engaged a team of six students from Rensselaer Polytechnic Institute's ITWS 4100 Information Technology Capstone course to design, develop, and deliver a unified search engine that leverages Natural Language Processing to help employees find relevant files quickly and intuitively.

The goal of the project is to create an intelligent search experience that understands user intent, delivers accurate results, and improves productivity across the organization. The project brief specified four deliverables: defined use cases and requirements based on interviews and research, market research on NLP security and boundaries for handling sensitive company data, technical architecture documentation addressing business logic and compliance, and a functioning web application prototype enabling natural language search across provided resources.

This report documents the full scope of the team's efforts and results over the course of the Spring 2026 semester. It covers the client organization and the problem being addressed, the team's composition and areas of expertise, the strategic analysis that informed the solution approach, the system's technical design and implementation, the financial and cost-benefit analysis, the project management plan and timeline, and the results achieved. The report also includes a post-turnover plan for education, training, maintenance, and ongoing support.

Section 3: Client Organization & Description

Deloitte Touche Tohmatsu Limited, commonly known as Deloitte, is one of the world's largest professional services organizations. The firm reported $70.5 billion in aggregate global revenue for fiscal year 2025, a 4.8% increase in local currency from the prior year (Deloitte Global, 2025). Deloitte employs over 470,000 people across more than 700 offices in over 150 countries, making it the largest of the "Big Four" professional services firms by both revenue and headcount.

Deloitte operates across four primary service lines: Audit and Assurance, Consulting, Financial Advisory, and Tax and Legal. The firm's workforce is knowledge-intensive by nature. Consultants, auditors, tax advisors, and specialists depend on internal documents, policies, methodologies, past deliverables, and training materials to serve their clients effectively. This dependence on internal knowledge makes the quality of information retrieval a direct driver of workforce productivity and, ultimately, billable utilization.

The firm has made significant investments in artificial intelligence and digital transformation. Deloitte launched PairD, an internal generative AI assistant, and has committed 36% of its digital budget to AI-related initiatives. The Deloitte AI Institute and the Omnia AI platform represent sustained organizational investment in AI research and enterprise deployment. This institutional commitment to AI adoption provided a natural foundation for the search engine project: the organization already has the cultural readiness, technical infrastructure, and leadership support for AI-driven internal tooling.

At Deloitte's scale, even modest improvements in operational efficiency translate into material financial impact. With 470,000 employees, each recovering even a fraction of the estimated 2.8 hours per week lost to information search, the cumulative productivity gains are substantial. The enterprise search market itself is valued at $7.47 billion in 2026 and projected to reach $11.66 billion by 2031 at a compound annual growth rate of 9.31% (Mordor Intelligence, 2026), reflecting the broader industry recognition that intelligent search is no longer a convenience but foundational infrastructure for knowledge-intensive organizations.

Throughout the project, the team coordinated with Deloitte stakeholders through the course's structured engagement process. Feedback sessions were held at key milestones to validate requirements, review prototype progress, and ensure alignment with Deloitte's expectations for the deliverables.

Section 4: Project Team

The project team consisted of six students at Rensselaer Polytechnic Institute, each with specific roles tailored to their areas of expertise. The team was assembled to cover the three disciplines this project demands: NLP-powered retrieval systems, AI safety and compliance, and user-centered design research.

Raven Levitt served as the Project Manager. Raven is a Co-founder of Meridian, a software platform serving thousands of active users, where he has managed the full product lifecycle from concept to launch. He has also served as a Vice President at HackRPI and as a leader within RCOS, directing teams ranging from agile squads of five to large-scale operations of over 100. For this project, Raven was responsible for coordinating across all workstreams, managing the nine-week timeline, tracking milestones, and ensuring that development, research, and design efforts converged on schedule. His operational experience allowed him to translate business requirements into technical reality while keeping the project on track and aligned with the strategic vision.

Felix Tian served as the System Architect. Felix is a senior Information Technology & Web Science student and the lead developer of the FinGPT Search Agent project hosted at Columbia University. He has published a peer-reviewed paper, "Customized FinGPT Search Agents Using Foundation Models" (arXiv:2410.15284), proposing an institutional retrieval-augmented search system designed for numerical accuracy, reduced hallucination,

and air-gapped deployment. Much of the architectural groundwork for the Deloitte search engine, including the hybrid retrieval pipeline, embedding strategy, and reranking approach, draws directly from Felix's prior research, providing the team with a meaningful head start. Felix was responsible for all major architectural decisions, the search pipeline design, embedding model selection and deployment, and integration of the Cohere Rerank layer.

Sophia Turnbow served as the User Researcher. Sophia is a senior dual-majoring in Information Technology & Web Science and Sustainability Studies, with a strong background in user experience design and user research. Her experience spans information discovery, workflow optimization, and user-centered system design across academic and applied projects. She has led qualitative research efforts including interview design, user recruitment, usability testing, and insight synthesis for data-driven digital products. As Chief Marketing Officer at Project Almanac, Sophia applies user research and systems thinking to guide product positioning and adoption strategy. For this project, Sophia designed and conducted the stakeholder interview protocols, developed user personas, planned and executed usability testing, and led UI/UX iteration based on user feedback.

Matthew Voynovich served as the Security Researcher. Matthew is a dual major in Computer Science and Information Technology & Web Science with a concentration in AI, machine learning, and data science. His research focuses on AI agent workflow safety, privacy leakage, and policy compliance in agent orchestration frameworks such as LangChain and LangGraph. This work directly informed the security guardrails built into the search engine, including input validation, prompt injection defenses, and the architecture of the role-based access control system. Matthew also has contract experience as a full-stack developer, delivering production web applications using React, Vue.js, Node.js, TypeScript, and PostgreSQL. For this project, Matthew led the security architecture, co-authored the NLP Security and Market Research report, and implemented the system's input validation and content filtering layers.

Jesse Gabriel served as a Developer. Jesse is a senior pursuing a dual major in Computer Science and Information Technology & Web Science with a concentration in Information Security, as well as a minor in Business Management. As a former Cyber Analyst intern at Deloitte within the Government and Public Services practice, Jesse brings direct experience navigating Deloitte's internal systems, compliance expectations, and consulting workflows. This firsthand knowledge grounded the team's design decisions in real operational context, ensuring the search engine reflects how Deloitte professionals actually search for and use internal resources. For this project, Jesse contributed to backend development, API endpoint implementation, Docker and deployment infrastructure, and security testing. His security background also informed the system's data handling and access control design.

Andrew Jung served as the Market Researcher. Andrew is a senior Information Technology & Web Science major with an Economics minor. As a co-founder and Frontend Lead at TandmAI, an AI governance startup focused on preventing employees from leaking sensitive and proprietary information through generative AI tools, Andrew worked closely with policy enforcement, prompt classification, and role-based access controls designed to safeguard PII and confidential company data. For this project, Andrew led the market

research on NLP security, privacy, and query boundaries, evaluating how modern AI-powered search systems restrict access, handle sensitive data, and enforce safe query behavior. His work translates real-world enterprise risks and best practices into actionable system requirements, directly informing the security and governance design of the search engine.

Section 5: Problem Statement

Deloitte employees rely on a variety of internal sites and platforms to access company documents, policies, slide decks, training materials, and images. These resources are scattered across multiple systems, and searching for a specific item often requires navigating several tools, remembering exact titles or filenames, and knowing which platform a particular resource lives on. When employees cannot clearly describe what they need, or when the resource they are looking for uses different terminology than their query, traditional keyword-based search fails. The result is wasted time, duplicated effort, and frequent reliance on colleagues to locate information manually.

The scale of this problem is well-documented in industry research. According to APQC (2021), knowledge workers spend an average of 2.8 hours per week searching for or requesting information they need to do their jobs, contributing to a 25% loss of the average workweek to productivity drains. A separate study by the Microsoft Work Trend Index (2023) found that 62% of workers report spending too much time searching for information during their workday. Despite this, only 27% of companies have implemented effective search tools to address the issue (Slite, 2025).

The financial impact at Deloitte's scale is impactful. Professional services billable utilization averaged 68.9% in 2024, below the 70–75% optimal threshold where profitability is maximized (SPI Research, 2025). Every hour a consultant spends searching for a document is an hour that is not billed to a client. With 470,000 employees, even a conservative estimate of the productivity loss represents billions of dollars in unrealized billable capacity annually. Beyond the direct financial cost, the problem creates downstream effects: consultants assemble deliverables from outdated or incorrect document versions, junior employees escalate simple document location requests to senior colleagues, and institutional knowledge remains siloed in individual employees' memory rather than being discoverable across the organization.

The root cause is not a lack of documents. Deloitte has decades of consulting engagements, audit reports, policy documents, methodologies, and training materials stored across its internal systems. The problem is that this knowledge is not accessible through a single, intelligent interface. Employees are forced to search across multiple platforms using exact keywords, and when those keywords do not match the terminology used in the document, which is a common occurrence in a large, diverse organization, the search returns nothing useful. There is no semantic understanding, no ability to interpret user intent, and no unified retrieval layer that connects all of the organization's resources.

This is the problem the project addresses: building a search engine that understands natural language, interprets what the user is looking for rather than just matching keywords, and returns relevant results from across the organization's internal resources, regardless of which platform those resources are stored on or what exact terminology they use.

Section 6: Strategy & Outline for the Solution (Mini-Strategic Plan)

6.1 Industry Definition and Market Size

This project is positioned within the enterprise search software industry. Enterprise search platforms are purpose-built systems that index, retrieve, and rank internal organizational content using techniques such as semantic search, keyword search, and neural reranking. The market was valued at $7.47 billion in 2026 and is projected to reach $11.66 billion by 2031, reflecting strong growth in demand for tools that help knowledge workers locate internal information efficiently.

For this analysis, the competitive set excludes general productivity suites with embedded search features, such as Notion AI, Slack AI, and Microsoft 365 Copilot. While those tools provide limited search functionality, they are not designed to serve as full enterprise search platforms across heterogeneous repositories, document types, and role-based permission structures. Deloitte's environment requires a purpose-built retrieval system capable of indexing diverse internal resources while enforcing access controls across a large and complex organization. In that context, enterprise search should be understood as a dedicated organizational capability rather than a feature inside a broader collaboration suite.

6.2 Five Forces Analysis

The enterprise search industry is attractive because demand is durable and tied to persistent organizational inefficiencies. Knowledge workers spend substantial time searching for information, and that inefficiency does not disappear during economic downturns because it is rooted in organizational complexity. At the same time, recent advances in retrieval-augmented generation, embeddings, and generative AI have made search systems more capable and strategically important.

However, the competitive forces in the industry are intense. **The threat of new entrants is high** because open-source retrieval frameworks, embedding models, and cloud infrastructure have lowered the cost of building functional search systems. **The power of suppliers is low to moderate** because cloud computing is commoditized and many

embeddings and retrieval components are open source, although some specialized APIs such as rerankers still create pockets of dependency. **The power of buyers is high** because large enterprises are sophisticated purchasers and can build internal alternatives if commercial pricing becomes excessive. **The threat of substitutes is moderate**, primarily from bundled search tools inside major productivity ecosystems, which may be "good enough" for less complex organizations. Finally, **the intensity of rivalry is high**, with well-funded firms such as Glean, Elastic, Coveo, and Microsoft continuously improving their products in a fast-cycle market.

6.3 Competitive Landscape

The most relevant competitors in the enterprise search space are Glean, Elastic, Microsoft Search, Coveo, and Algolia. Among these, Glean and Microsoft Search are the most direct alternatives because they target large organizations looking to improve internal knowledge retrieval. Glean is a purpose-built AI enterprise search platform with strong application integrations and personalization capabilities, but it is expensive and introduces vendor lock-in. Microsoft Search benefits from existing Microsoft ecosystem adoption and low marginal cost for current customers, but it is constrained by limited file-type support and dependence on the Microsoft stack.

Elastic offers strong flexibility and customization through an open-source foundation, but it is better understood as search infrastructure than as a finished enterprise product. Coveo is strong in customer-facing and commerce-oriented search use cases, but is less directly aligned to Deloitte's internal knowledge retrieval needs. Algolia is fast to implement and developer-friendly, but is primarily optimized for more conventional search use cases and offers less depth in semantic enterprise retrieval. Overall, the competitive analysis suggests that while commercial platforms are strong, none fully match the combination of Deloitte-specific content, workflow alignment, access control requirements, and data sovereignty that an internal solution can provide.

6.4 Internal Resources and Capabilities

Deloitte's internal position strongly supports an internal search initiative. The firm reported $70.5 billion in revenue for FY2025, employs more than 470,000 people, and operates across a globally distributed, knowledge-intensive workforce. This creates both the need for better information retrieval and the organizational capacity to build and support such a system.

From a resource perspective, Deloitte has substantial tangible assets, including financial capacity, enterprise-grade infrastructure, and a massive corpus of internal documents accumulated over decades. It also has critical intangible resources, including professional-services domain expertise, an established innovation culture through the Deloitte AI Institute and Omnia AI, and a reputation that demands strict handling of

confidential client and firm information. These resources combine into valuable organizational capabilities: in-house AI and engineering talent, mature data governance practices, and experience deploying applications across a large global organization.

Within Deloitte's value chain, the search engine functions as both a product and a support capability. Its value-creating activities include document ingestion, embedding generation, hybrid retrieval, reranking, access control filtering, interface delivery, and feedback-based search improvement. Because the system directly improves access to organizational knowledge, it also serves as a management information systems capability that supports Deloitte's broader consulting, audit, tax, and advisory work.

6.5 VRIN Analysis

The VRIN analysis shows that Deloitte's sustainable competitive advantage comes primarily from its proprietary internal document corpus, not from the search technology alone. The corpus is valuable because it enables faster and more relevant retrieval of internal knowledge. It is rare because no outside organization has access to Deloitte's specific body of historical deliverables, policies, methodologies, and training materials. It is costly to imitate because it was created over decades through unique firm activity, and it is nonsubstitutable because no external knowledge base can replace the value of Deloitte's own institutional content. This makes the corpus a true source of sustainable advantage.

Other resources are important but less defensible. Deloitte's domain expertise in professional-services search provides a temporary advantage, but competing firms could develop similar expertise over time. Its RBAC and compliance infrastructure is necessary, but broadly available across enterprises and therefore a source of parity rather than differentiation. The hybrid search pipeline and open-source technology stack are also strategically useful, especially because they reduce vendor dependence, but they do not provide a durable moat because similar architectures can be replicated by others. The central strategic insight is that the technology is the delivery mechanism, but the data is the moat.

6.6 Business-Level Strategy: Focused Differentiation

Our recommended business-level strategy for this initiative is focused differentiation. The scope is narrow by design: this is not a product for the general market, but an internal system tailored to the needs of Deloitte's workforce. Within that narrow scope, the objective is not simply to minimize cost, but to deliver a search experience that is meaningfully superior for Deloitte users because it is tuned to their documents, vocabulary, organizational structure, and security requirements.

This differentiation comes from three sources. First, the search engine can be optimized for professional-services work, including Deloitte-specific document types and search

behavior patterns. Second, it can enforce access controls that reflect Deloitte's internal hierarchy and governance model. Third, it preserves complete data sovereignty by keeping sensitive internal and client-related materials under Deloitte's control rather than routing them through a third-party platform. Although the open-source stack also produces major cost advantages, those savings are a byproduct of architectural choices rather than the primary strategic objective. The strategic priority is distinctiveness and fit, not generic low-cost leadership.

6.7 Build vs. Buy Rationale

The build-versus-buy decision strongly favors building internally. Commercial enterprise search platforms can provide usable functionality, but at Deloitte's scale their recurring licensing costs become enormous. For example, a platform priced at approximately $20 per user per month would cost more than $112 million annually across Deloitte's workforce. By contrast, the team's architecture is built on an open-source-first stack that avoids per-user licensing fees and reduces long-term vendor dependency.

Just as important, an internal build better matches Deloitte's strategic needs. A custom system can be tuned for the firm's own vocabulary, document types, access-control model, and workflow patterns in ways that a commercial platform cannot fully replicate. It also ensures that highly sensitive internal and client-related information never leaves Deloitte-controlled infrastructure. In strategic terms, building the system is a high-commitment action, but one that creates durable internal value through better organizational fit, stronger governance, and greater control over a core information systems capability.

6.8 Strategic Risks

The focused differentiation strategy does carry meaningful risks. One risk is out-focusing, where a more specialized solution for a single service line or use case could outperform a firm-wide search tool within that narrower segment. This risk can be mitigated by designing the architecture to support modular tuning for different service lines without fragmenting into entirely separate systems.

A second risk is that a major platform vendor, especially Microsoft or Google, could improve embedded enterprise search enough that a dedicated internal tool appears less necessary. A third risk is that Deloitte's own information environment could become more standardized over time, reducing the value of a specialized solution if documents and workflows converge into a single platform with strong native search. These risks do not invalidate the strategy, but they do establish its boundary conditions. To remain superior, Deloitte would need to continue benchmarking commercial alternatives, evolving the system's relevance quality, and preserving the advantages created by proprietary data, domain tuning, and organizational integration.

Section 7: Analysis Methodology

7.1 Development Methodology

The project followed an Agile development methodology organized around a nine-week timeline aligned with the Spring 2026 semester. The team used Kanban boards and Calendar view in Jira to manage tasks, track progress, and visualize workstream dependencies across development, research, and design efforts. GitHub served as the central version control platform, Google Drive supported document collaboration across all team members, and Discord facilitated daily communication and coordination.

Development progressed through nine sequential phases, each focused on a distinct layer of the system: (1) project infrastructure and Docker Compose orchestration, (2) database schema and index initialization, (3) backend API scaffolding with FastAPI, (4) external embedding service deployment, (5) document ingestion pipeline with Docling, (6) hybrid search pipeline with Reciprocal Rank Fusion, (7) frontend implementation with Next.js, (8) admin upload portal, and (9) integration testing with demo data. This phased approach ensured that foundational infrastructure was stable before higher-level application logic was built on top of it, reducing the risk of cascading failures during integration.

The team coordinated with Deloitte stakeholders through structured weekly meetings to provide progress updates, gather feedback, and validate requirements at each milestone. This regular feedback loop allowed the team to adjust priorities based on client input. For example, following a stakeholder meeting on March 12, 2026, the team deprioritized the admin upload portal in favor of refining the core search experience, based on Deloitte's guidance that documents would be pre-loaded sample data rather than user-uploaded content in the prototype.

AI-assisted development tooling, specifically Claude Code (Anthropic's agentic coding CLI), was used throughout the implementation phases to accelerate development velocity. The system architect used Claude Code for architectural scaffolding, code generation, debugging, and documentation, enabling a small team to deliver a full-stack prototype within the compressed semester timeline. All AI-generated code was reviewed, tested, and validated by team members before integration.

7.2 User Research

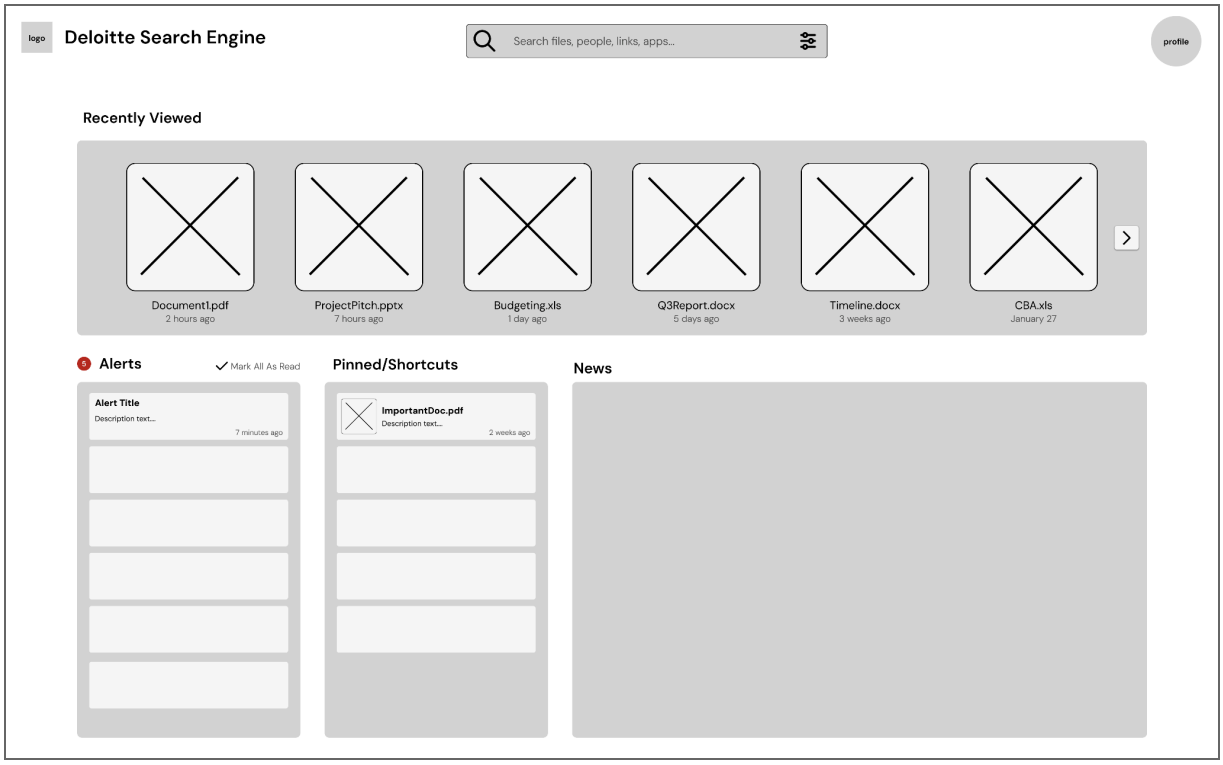
The user research component of this project was conducted in parallel with system design and prototyping, with the goal of grounding interface decisions in real user needs and workflows. Due to access limitations at the company, direct stakeholder interviews and large-scale usability testing were not feasible. Instead, a hybrid approach was employed, combining iterative feedback from client meetings with a structured user survey distributed

to client representatives. This methodology enabled the collection of both qualitative and experience-based insights from individuals very familiar with Deloitte's internal tools and document ecosystems.

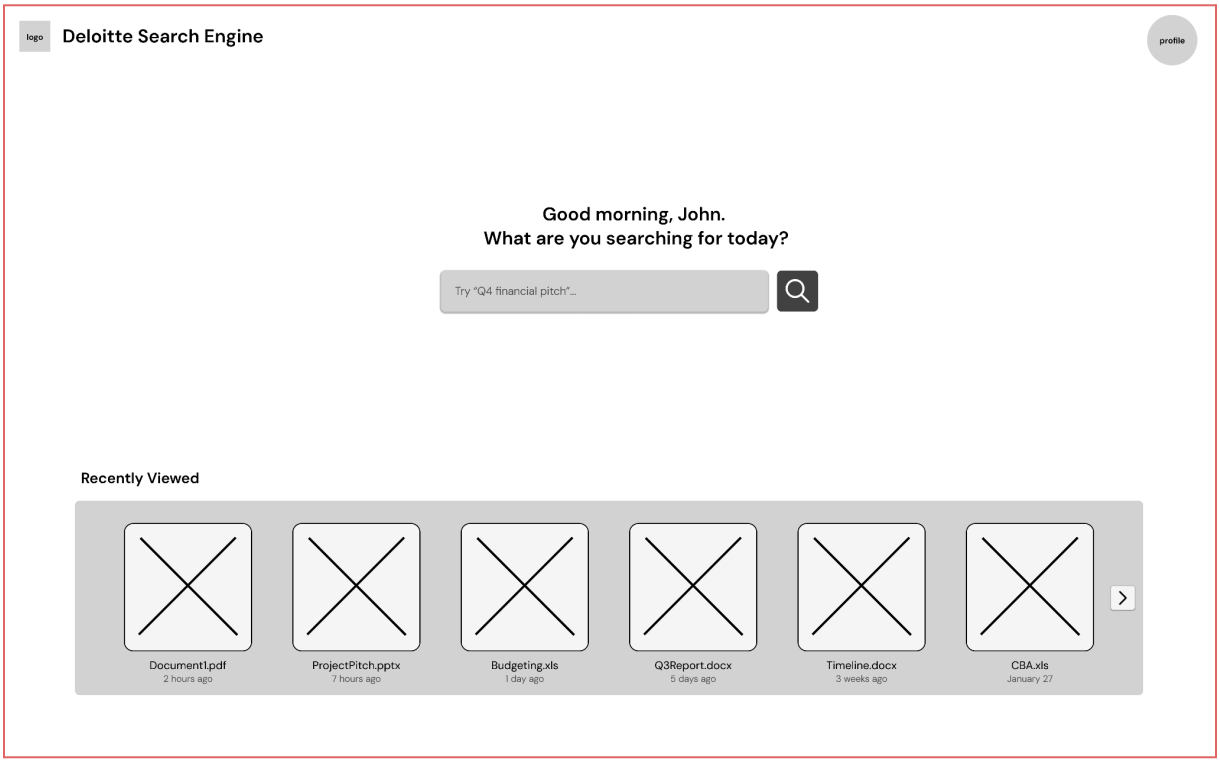
The research was designed to capture a range of perspectives across service lines, including consulting, tax, and audit, in order to reflect the diversity of search behaviors within the organization. The survey focused on several key objectives: understanding how users currently interact with internal search systems, identifying common pain points and inefficiencies, determining the types of documents most frequently searched for, and evaluating preferences for how search results and metadata should be presented.

Responses revealed several consistent themes. Users frequently described existing search systems as overwhelming, citing the presence of large volumes of irrelevant or outdated results. A recurring issue was the difficulty in identifying the correct document, even when it appeared within the results, due to a lack of visual distinction or prioritization. Recency emerged as a critical factor in user trust, with outdated documents significantly reducing confidence in the system. Additionally, users expressed frustration with inefficient workflows, such as the need to open multiple browser tabs, and challenges associated with legacy naming conventions following organizational restructuring.

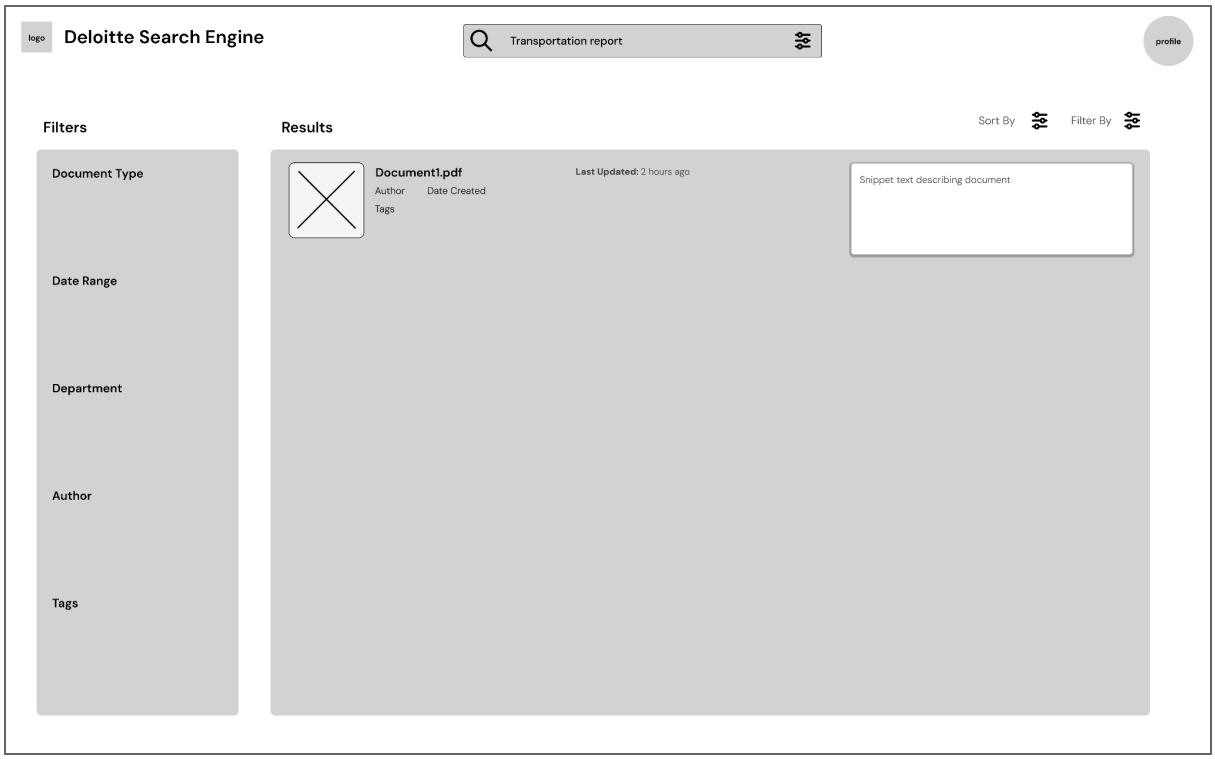
To address these findings, the design process incorporated an iterative approach beginning with low-fidelity wireframes. Two initial design directions were explored. The first followed a traditional dashboard-style layout, incorporating multiple panels for saved items, recent documents, and news *(7.2.1 Homepage Version A)*. This version was based on the example layout they gave us that referred to their current search system. The second adopted a more minimalist, search-focused interface centered around a single primary interaction *(7.2.2 Homepage Version B)*. These early mockups were intentionally simplified in wireframe format to evaluate layout and usability without the influence of detailed visual styling.



7.2.1 Wireframe Homepage Version A

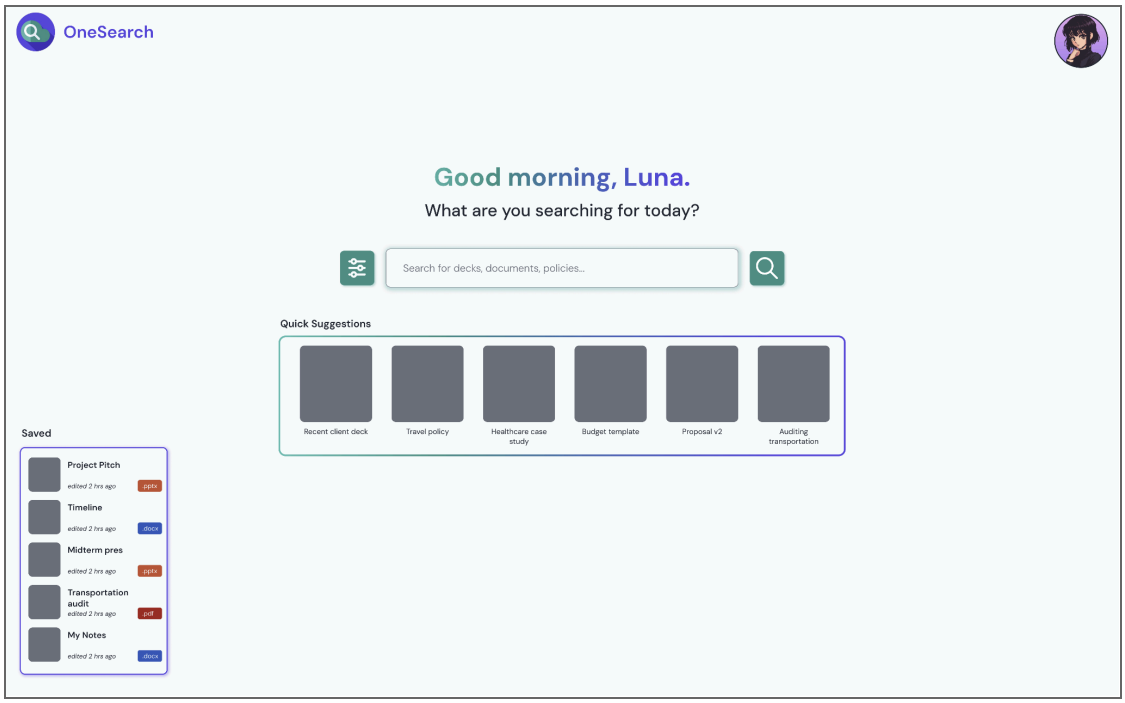


7.2.2 Wireframe Homepage Version B

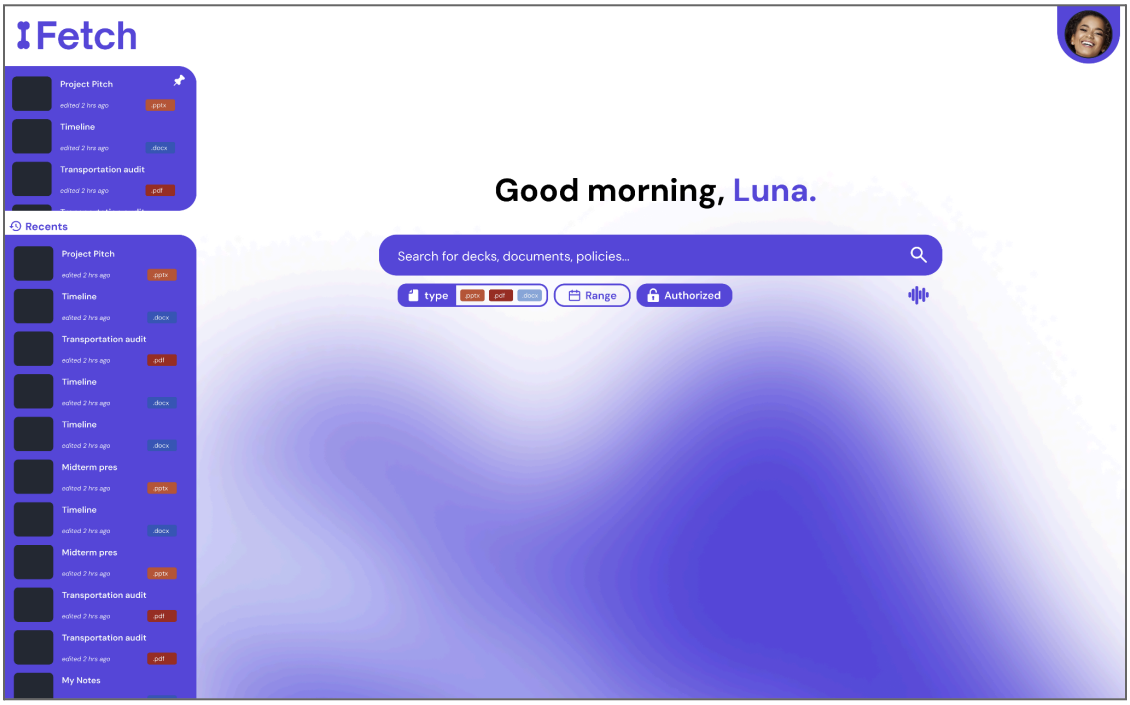


7.2.3 Wireframe Search Results

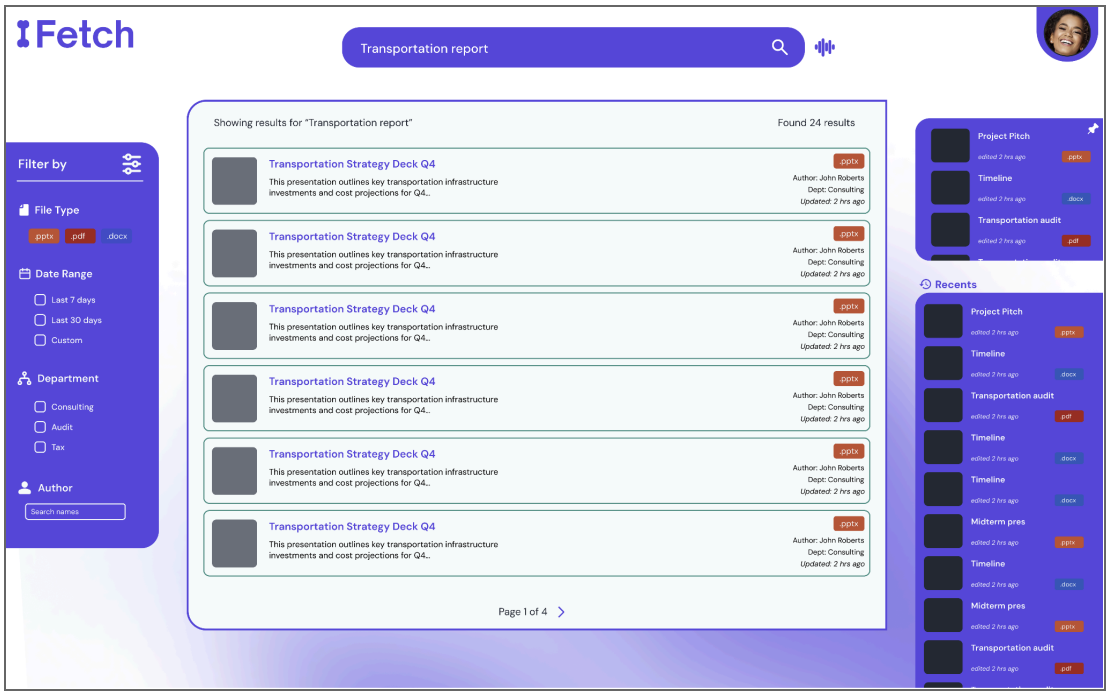
The clients strongly favored Version B, the minimalist direction, due to its clarity, reduced cognitive load, and alignment with modern search expectations. However, elements from the dashboard design—specifically recently viewed items and pinned or saved documents—were identified as valuable features and were incorporated into the subsequent iterations. These features were integrated into the main content flow rather than isolated in separate panels, improving cohesion and usability. We explored a couple different design directions from here *(Mockups 7.2.4, 7.2.5, 7.2.6)*, and later received input from the clients during our next meeting and from the survey feedback.



7.2.4 Homepage Mockup Version 1

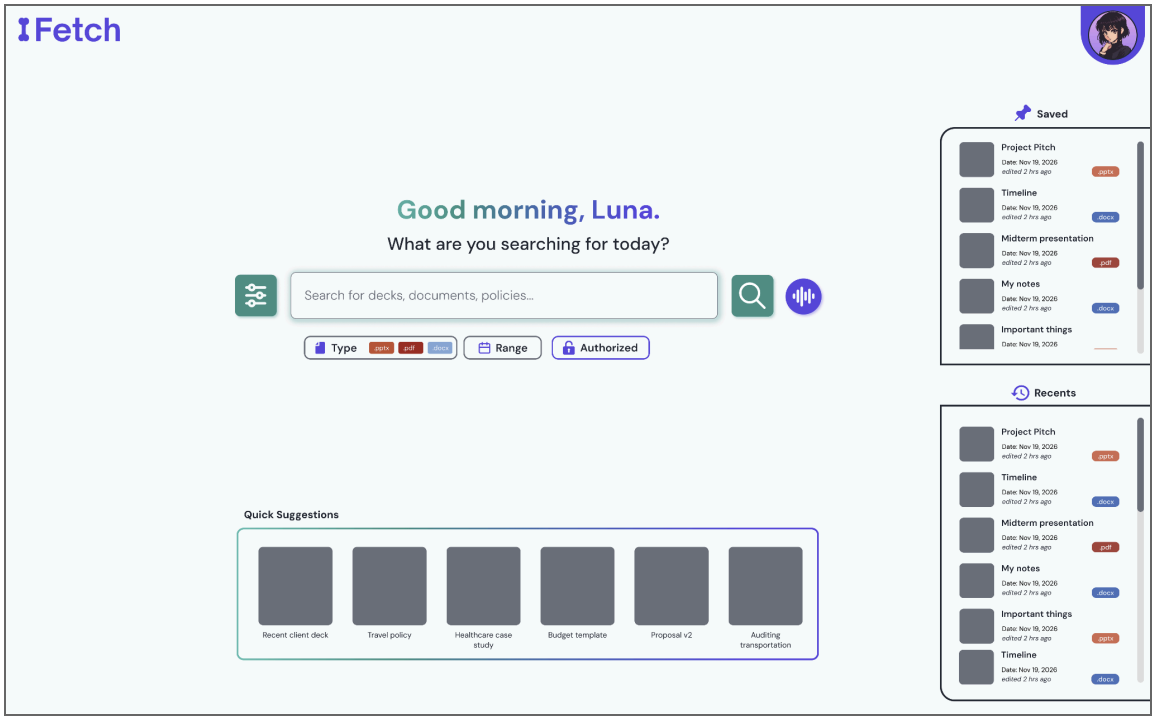


7.2.5 Homepage Mockup Version 2

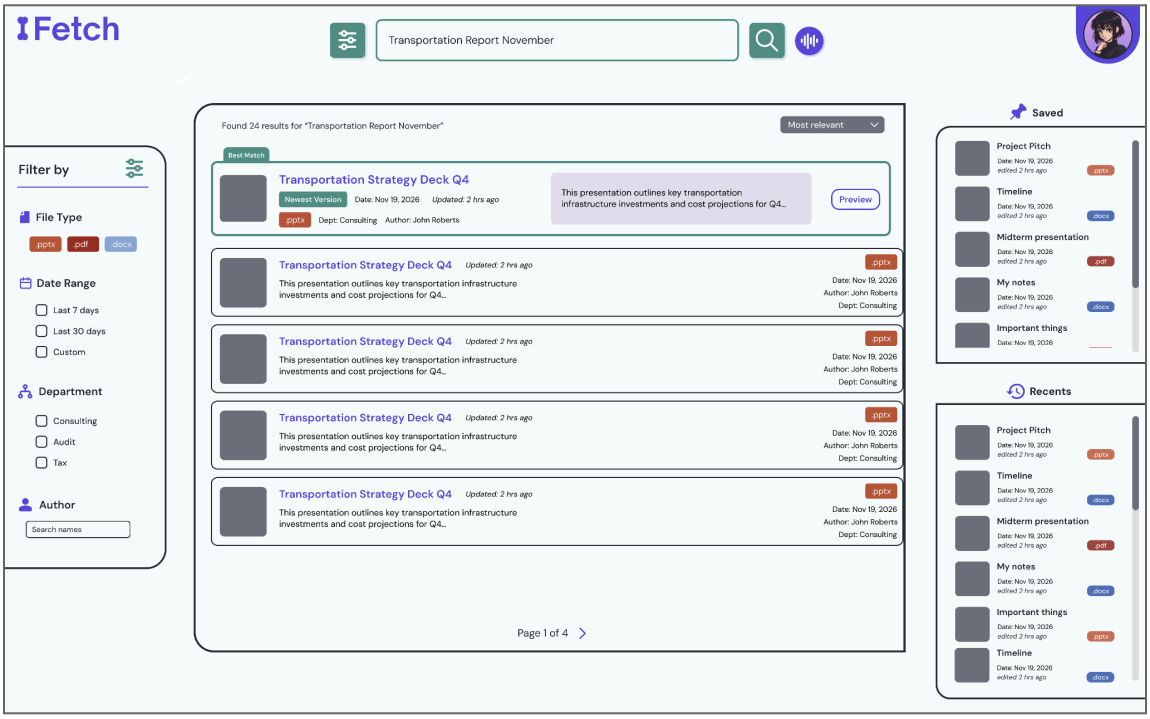


7.2.6 Search Results Mockup Version 2

Further refinement of the design was informed directly by survey responses. For example, the emphasis on recency led to the prioritization of "last updated" metadata in search results, while concerns about irrelevant content influenced the introduction of visual hierarchy techniques such as highlighting the most relevant result *(7.2.8)*. Feedback regarding visual clutter also guided the reduction of excessive color and the adoption of a cleaner, more restrained interface.



7.2.7 Homepage Mockup FINAL



7.2.8 Search Results Mockup FINAL

In addition to informing interface structure, user research also supported the development of representative personas, capturing variations in experience level, departmental needs, and search behaviors. These personas were used to validate design decisions and ensure that the system addressed a broad range of user requirements across the organization.

Overall, the user research process, while constrained in scope, provided actionable insights that directly shaped both the functionality and presentation of the system. By integrating feedback iteratively throughout the design process, the final interface reflects a deliberate alignment between user needs and system capabilities.

7.3 Market Research

The market research focused on determining where the proposed Deloitte search engine fits within the enterprise search market and what strategic advantages an internal solution could offer. To do this, the team combined industry sizing, competitor review, and strategic analysis frameworks from *Strategic Management: Concepts and Cases* by Hitt, Ireland, and Hoskisson.

The first step was defining the market and narrowing the competitive set. Using published market data, the team identified enterprise search as a growing software category and distinguished purpose-built enterprise search platforms from general productivity tools such as Slack AI, Notion AI, and Microsoft Copilot. Those tools were excluded from the main competitive set because they do not provide the same breadth of retrieval, repository coverage, or access-control support needed for a Deloitte-scale internal knowledge system.

The next step was evaluating industry conditions through Five Forces analysis. This was used to assess how attractive the enterprise search market is and what pressures shape competition. The analysis found that the space is attractive because demand for knowledge retrieval is persistent, but also highly competitive because open-source tools have lowered entry barriers, enterprise buyers have strong bargaining power, and rivalry among major vendors is intense. The team's own experience building a hybrid retrieval prototype also informed this assessment by showing how much functionality can now be built with low-cost open-source components.

The team then conducted a competitor analysis of major players including Glean, Elastic, Coveo, Microsoft Search, and Algolia. This comparison examined market overlap, technical positioning, strengths, and weaknesses. It showed that Glean and Microsoft Search are the closest alternatives for Deloitte's use case, but also highlighted tradeoffs such as vendor lock-in, limited file-type support, and high recurring licensing costs at Deloitte's scale.

Finally, the market research connected the external analysis to Deloitte's internal position. Using VRIN and related strategic analysis, the team concluded that the project's long-term advantage would not come from the search technology alone, since hybrid retrieval

methods can be replicated, but from Deloitte's proprietary internal document corpus, domain-specific workflows, and existing governance infrastructure. This led to the conclusion that the most appropriate strategic framing for the project is focused differentiation: an internal system designed specifically for Deloitte's workforce, tuned to professional-services search behavior, and built to preserve complete data sovereignty.

7.4 Financial Analysis Approach

Our financial analysis followed a structured cost-benefit methodology to evaluate the economic viability of deploying the search engine within Deloitte. The analysis modeled a 50-person pilot cohort over a three-year projection horizon, using conservative assumptions to establish a credible baseline for decision-making.

The cost model quantified three categories of expenditure: development costs (team effort over the project period), infrastructure costs (PostgreSQL hosting, embedding server compute, and Docker-based deployment), and ongoing maintenance costs (monitoring, updates, and model retraining). Because the technology stack is entirely open-source, the cost model reflects zero per-user licensing fees, which is the primary financial differentiator from commercial alternatives.

The benefit model centered on productivity recovery. Using APQC's (2021) finding that knowledge workers lose an average of 2.8 hours per week to information search, the analysis assumed a conservative recovery of 1.5 hours per week per adopter within a cohort where 70% of the 50-person pilot actively uses the system. Benefits were quantified using Deloitte's average professional services billing rates.

Three standard financial metrics were computed. Net Present Value (NPV) was calculated using a 10% discount rate, yielding a positive NPV of $289,505 over three years. The Internal Rate of Return (IRR) was approximately 221%, indicating that the project's return substantially exceeds the cost of capital. The payback period fell within the first year of operation.

A sensitivity analysis tested the model under worst-case, base-case, and best-case scenarios by varying key assumptions including adoption rate, hours recovered per user, and infrastructure costs. All three scenarios produced a positive NPV, confirming that the project's financial viability is robust to reasonable variation in input assumptions.

A risk assessment identified and quantified potential risks using expected monetary value (EMV). The total risk exposure was assessed at $16,600 in EMV, well within manageable bounds for a pilot-scale deployment. Key risks included adoption shortfall, embedding model performance degradation, and infrastructure cost overruns. Each risk was paired with a mitigation strategy and probability estimate.

The build-versus-buy comparison provided additional strategic context. At Deloitte's full scale of 470,000 employees, a commercial alternative such as Glean at $20 or more per

user per month would cost over $112 million annually. The open-source approach eliminates this recurring cost entirely, making the total cost of ownership a fraction of commercial alternatives even when accounting for internal development and maintenance effort.

Section 8: IS/IT Design (Requirements, Code, Screenshots, Database)

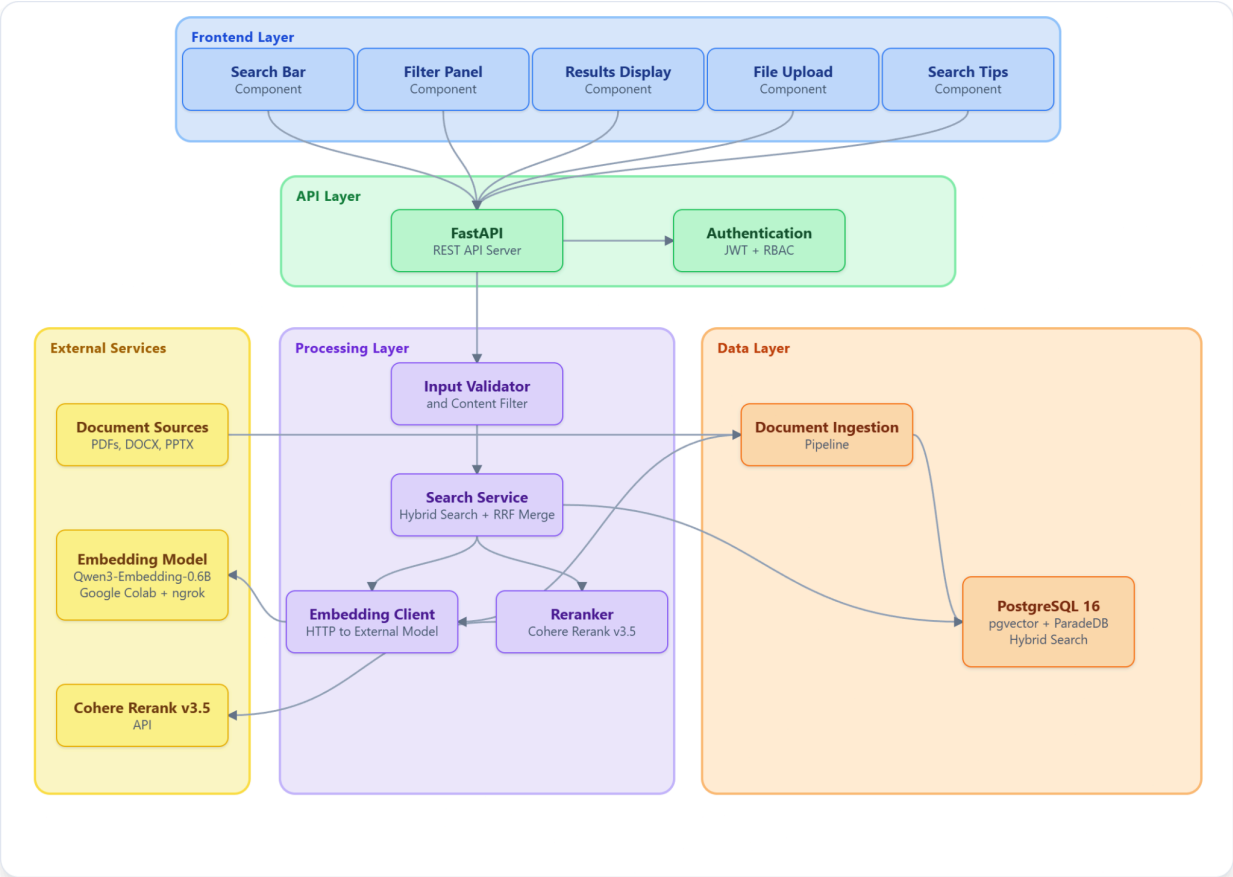
8.1 System Requirements

This system was designed to satisfy both functional and non-functional requirements derived from Deloitte's project brief and validated through client feedback sessions. The core functional requirements include: (1) natural language search, allowing employees to query internal resources using conversational phrasing rather than exact keywords; (2) multi-format document ingestion, supporting PDF, DOCX, and PPTX files through a unified parsing pipeline; (3) role-based access control (RBAC), restricting document visibility based on user role (analyst, manager, or admin) and document classification (public, internal, or confidential); (4) document versioning, enabling the system to recognize and manage multiple versions of the same document while surfacing the most current version by default; and (5) audit logging, recording all search queries with associated metadata for analytics and compliance purposes.

Non-functional requirements establish the system's quality attributes. The search pipeline targets sub-two-second end-to-end response times for 95% of queries, balancing retrieval depth (50 candidates per signal) against latency constraints. The architecture supports horizontal scalability through stateless API and processing layers, with the database as the sole stateful component. Graceful degradation is built into every external dependency: if the Cohere Rerank API is unavailable, the system returns results in their Reciprocal Rank Fusion order; if the embedding service is unreachable, the system returns an informative error rather than failing silently. The system is designed for one-command deployment via Docker Compose, requiring no manual configuration beyond environment variables.

8.2 Technical Architecture

The system follows a four-layer architecture that separates concerns across the frontend, API, processing, and data layers. Figure 8.1 illustrates the component diagram for the complete system.



The Frontend Layer is built on Next.js 16 with TypeScript and the App Router pattern. It provides three pages: a landing page with the search interface, a search results page with filtering and version controls, and an admin upload portal. The UI uses shadcn/ui components built on Tailwind CSS 4 for consistent styling.

The API Layer uses FastAPI, a high-performance Python ASGI framework, running on Uvicorn. It exposes four router groups under the /api prefix: health checks, authentication (login and token issuance), search (the core query endpoint), and document management (upload and metadata). Cross-origin resource sharing (CORS) is configured permissively for the prototype.

The Processing Layer contains the system's core business logic. The search service orchestrates the hybrid retrieval pipeline, combining three parallel retrieval signals through Reciprocal Rank Fusion and optional neural reranking. The ingestion service handles document parsing, text chunking, embedding generation, and database storage. The embeddings service acts as an HTTP client to the external Qwen3-Embedding-0.6B model,

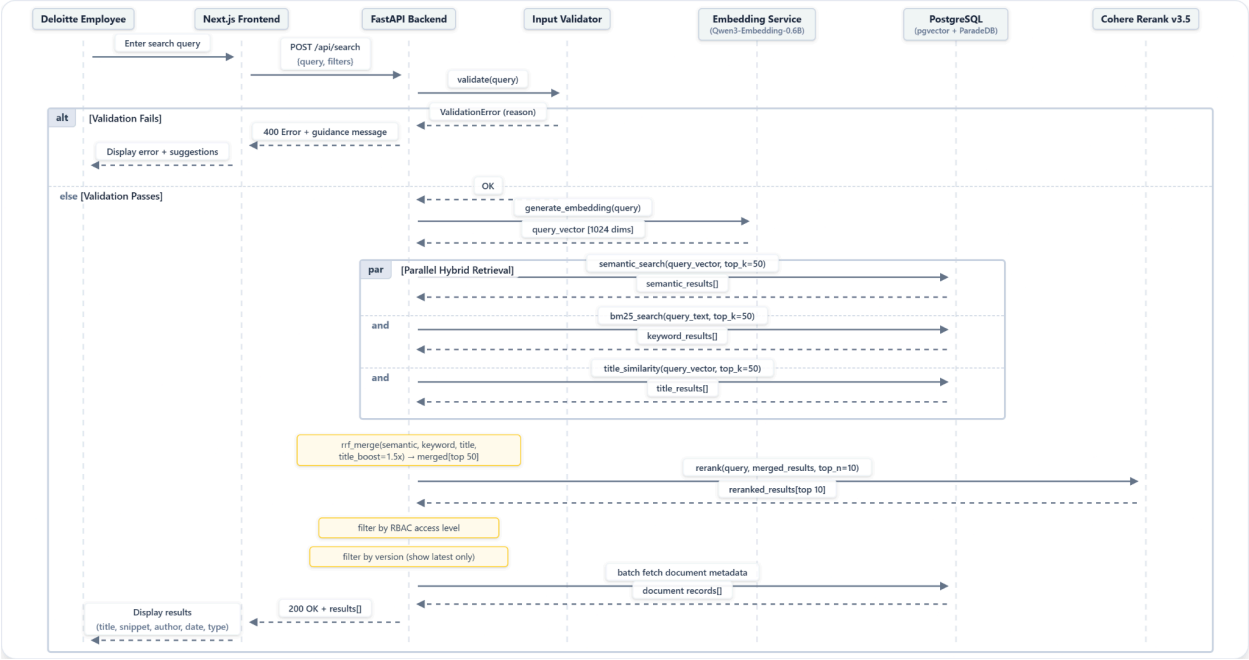
and the reranker service interfaces with the Cohere Rerank v3.5 API. A validation service enforces query length limits and blocks injection patterns.

The Data Layer centers on a single PostgreSQL 16 instance running the ParadeDB image, which bundles the pgvector and pg_search extensions. This unified database handles vector similarity search, BM25 keyword search, relational metadata storage, user authentication records, and query audit logs, eliminating the operational complexity of maintaining separate vector and relational databases.

External services include the Qwen3-Embedding-0.6B embedding model (deployed locally via embedding_server.py or remotely via Google Colab with ngrok tunneling) and the Cohere Rerank v3.5 neural reranker (accessed via API).

8.3 Search Pipeline Design

The search pipeline is the system's core differentiator: a hybrid retrieval approach that combines semantic understanding, keyword matching, and title similarity to produce results that neither approach could achieve independently. Figure 8.2 illustrates the complete query lifecycle as a sequence diagram.



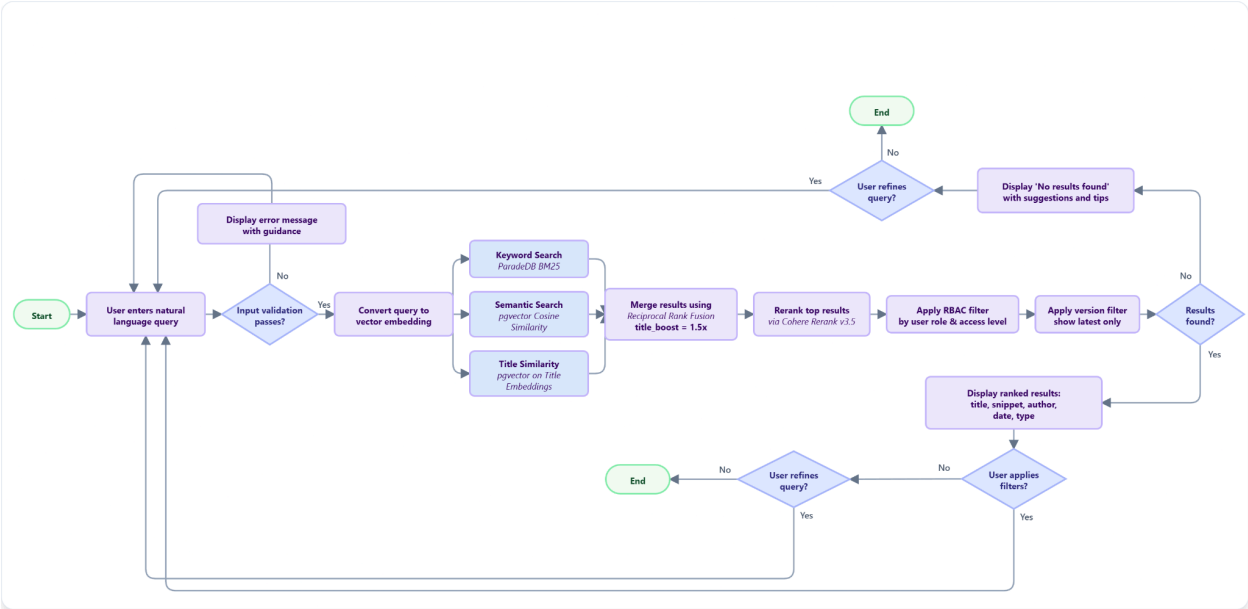
When a user submits a query, the system first validates the input against length constraints (maximum 500 characters) and injection pattern filters that block prompt injection attempts such as "ignore previous instructions" and script injection via HTML tags. The validated query is then sent to the external Qwen3-Embedding-0.6B model, which returns a 1024-dimensional vector representation capturing the query's semantic meaning.

The system performs three parallel retrieval operations against PostgreSQL. First, a semantic search computes cosine similarity between the query vector and all document chunk embeddings using pgvector's HNSW index, returning the top 50 most semantically similar chunks. Second, a BM25 keyword search uses ParadeDB's Tantivy-based full-text index to find the top 50 chunks with the strongest lexical overlap. Third, a title similarity search computes cosine similarity between the query vector and document title embeddings stored in a separate table with its own HNSW index, also returning up to 50 results. The BM25 query is sanitized prior to execution to remove Tantivy special characters that would cause parsing errors.

These three ranked lists are merged using Reciprocal Rank Fusion (RRF), a rank aggregation method that assigns each result a score of 1/(k + rank + 1), where k is a smoothing constant set to 60 (Cormack et al., 2009). Title matches receive a 1.5x weight multiplier in the RRF calculation, reflecting the empirical observation that documents whose titles match the query are disproportionately likely to be relevant. The merged candidates are then passed to the Cohere Rerank v3.5 neural cross-encoder, which re-scores each candidate against the original query and returns the top 10 results. A batch document metadata fetch resolves all document records in a single query to avoid N+1 performance degradation. Finally, version filtering (enabled by default) retains only the highest-versioned document within each document group, and RBAC filtering removes documents whose access level exceeds the user's role authorization. The entire pipeline is timed, and latency in milliseconds is returned alongside results.

8.4 Document Ingestion Pipeline

The ingestion pipeline transforms raw documents into searchable, embedded representations stored in PostgreSQL. Figure 8.3 illustrates the complete ingestion workflow as an activity diagram.



When a document enters the system, the pipeline first computes a SHA-256 hash of the file contents and extracts version information from the filename using a regex pattern that detects the _v<number> suffix (for example, "Remote_Work_Policy_v2.docx" maps to document group "remote work policy" with version 2). The system then checks for an existing record with the same document group and version. If one exists with an identical content hash, the document is skipped as a duplicate. If the content hash differs, the existing record is deleted (with cascading removal of its chunks and title embedding) before the updated document is processed.

The document is parsed using Docling (IBM), a unified parser that handles PDF, DOCX, and PPTX formats and outputs clean markdown text with structural awareness including tables and hierarchical headings. The extracted text is split into chunks of 512 words with a 50-word overlap between consecutive chunks. This word-based chunking strategy was chosen based on recent research (NAACL 2025) demonstrating that fixed-size chunks of approximately 200 words perform comparably to semantic chunking strategies while requiring significantly less computational overhead. The 512-word size was calibrated to balance between capturing sufficient context for analytical queries and maintaining embedding precision for specific factual lookups.

All chunks, along with the document title, are embedded in a single batch request to the Qwen3-Embedding-0.6B model. The first embedding (corresponding to the title) is stored in the document_title_embeddings table, while the remaining embeddings are stored alongside their text content in the document_chunks table. After the first document is

ingested, the system lazily creates the ParadeDB BM25 index on the document_chunks table, enabling keyword search for subsequent queries.

8.5 Database Design

The database schema comprises five tables designed around a single PostgreSQL 16 instance running the ParadeDB image. This unified approach stores relational metadata, 1024-dimensional vector embeddings, and full-text search indexes within a single database engine.

The documents table serves as the primary record for each ingested file, storing metadata (title, author, document type, category, creation date, page count), access control classification (public, internal, or confidential), and versioning information (document_group and version with a unique constraint ensuring no duplicate group-version combinations). A content_hash column (SHA-256) supports deduplication at ingestion time.

The document_chunks table stores the segmented text of each document alongside its vector embedding. Each chunk carries a chunk_index for ordering, a content field containing the raw text, and an embedding column typed as vector(1024). An HNSW index with parameters m=16 and ef_construction=200 using the vector_cosine_ops operator class enables approximate nearest neighbor search for semantic retrieval. The ParadeDB BM25 index on the content column, created dynamically after first ingestion, enables Tantivy-based keyword search.

The document_title_embeddings table stores a single 1024-dimensional embedding per document, representing its title in the same vector space as query embeddings. This separation from chunk embeddings allows the title similarity search to operate on its own HNSW index, which spans tens of rows rather than thousands, yielding faster lookup times and cleaner relevance signals for title-oriented queries.

The users table stores authentication credentials (bcrypt-hashed passwords) and role assignments (analyst, manager, or admin). The query_logs table records every search query with its user association, result count, selected document (if any), and latency in milliseconds, providing an audit trail for usage analytics and performance monitoring. Standard B-tree indexes on foreign keys (document_id, user_id) and frequently filtered columns (access_level, category, document_group) support efficient joins and filtered queries.

8.6 Security Architecture

The security architecture implements defense in depth across authentication, authorization, input validation, and audit logging. Authentication uses a standard JWT flow: users submit credentials to the /api/auth/login endpoint, which verifies the password

against a bcrypt hash (using the passlib library) and issues an HS256-signed JSON Web Token with a 60-minute expiry. The token payload contains the user's ID as the subject claim, and subsequent requests include this token in the Authorization header for identity verification.

Authorization follows a three-tier RBAC model that maps user roles to document access levels. Analysts can view only public documents. Managers can view public and internal documents. Administrators can view all documents, including those classified as confidential, and have access to the document upload interface. This filtering is applied at query time within the search pipeline: the SQL WHERE clause restricts results to documents whose access_level is in the set permitted by the requesting user's role. For the prototype demonstration, the search endpoint hardcodes the admin role to showcase the full document corpus.

Input validation acts as the first line of defense against adversarial queries. The validation service enforces a 500-character maximum query length and pattern-matches against known injection vectors including prompt injection phrases ("ignore previous instructions," "system prompt," "you are now") and script injection attempts ("<script" tags). Additionally, the BM25 query sanitizer strips Tantivy special characters to prevent query parser exploitation. All search queries are logged in the query_logs table with timestamps, user IDs, result counts, and latency, creating an audit trail that supports both usage analytics and forensic investigation in the event of a security incident.

8.7 Key Design Decisions

Several architectural decisions shaped the system's design, each involving explicit tradeoffs evaluated against the project's constraints of a small team, compressed timeline, and prototype-scale deployment.

The first and most consequential decision was adopting PostgreSQL as a unified database for vector search, keyword search, and relational storage. The alternative was to deploy separate specialized databases: Pinecone or Qdrant for vector search, Elasticsearch for keyword search, and PostgreSQL for metadata. While the separated approach would offer independent scalability for each workload, it would introduce inter-service network latency, metadata synchronization complexity, and significant operational overhead. The unified approach, enabled by the pgvector and ParadeDB extensions, reduces the deployment to a single database connection string and ensures that all data remains consistent without cross-database synchronization.

The second decision was the choice of Qwen3-Embedding-0.6B as the embedding model. Alternatives included OpenAI's text-embedding-3-large and Cohere's embed-v3, both of which offer strong benchmark performance but introduce per-request API costs and data sovereignty concerns, as all query text would be sent to external servers. Qwen3-Embedding-0.6B produces 1024-dimensional vectors with competitive retrieval

quality while running entirely under the team's control, either locally or on a private Colab instance. The 0.6B parameter count keeps inference fast even without GPU acceleration.

The third decision was implementing Docling (IBM) for document parsing rather than format-specific libraries like PyPDF2 or pdfplumber. Docling provides a unified API for PDF, DOCX, and PPTX formats with structural awareness (tables, headings, lists), eliminating the need to maintain three separate parsing codepaths and reducing the surface area for format-specific edge cases.

The fourth decision involved the hybrid search fusion strategy. Rather than a simple linear combination of semantic and keyword scores (which requires careful weight tuning and comparable score distributions), the team adopted Reciprocal Rank Fusion, which operates on rank positions rather than raw scores. RRF is parameter-light (requiring only the smoothing constant k), theoretically grounded (Cormack et al., 2009), and naturally handles the incomparable score scales between cosine similarity and BM25 relevance.

The fifth decision was to implement version filtering as a post-retrieval step rather than a pre-retrieval constraint. Filtering before search would require complex GROUP BY logic in the SQL queries and would prevent users from ever seeing older document versions. By filtering after ranking and reranking, the system preserves the option for users to toggle "show latest only" without re-executing the search pipeline, and it ensures that all versions contribute to the relevance scoring even when only the latest is displayed.

8.8 Frontend and User Experience

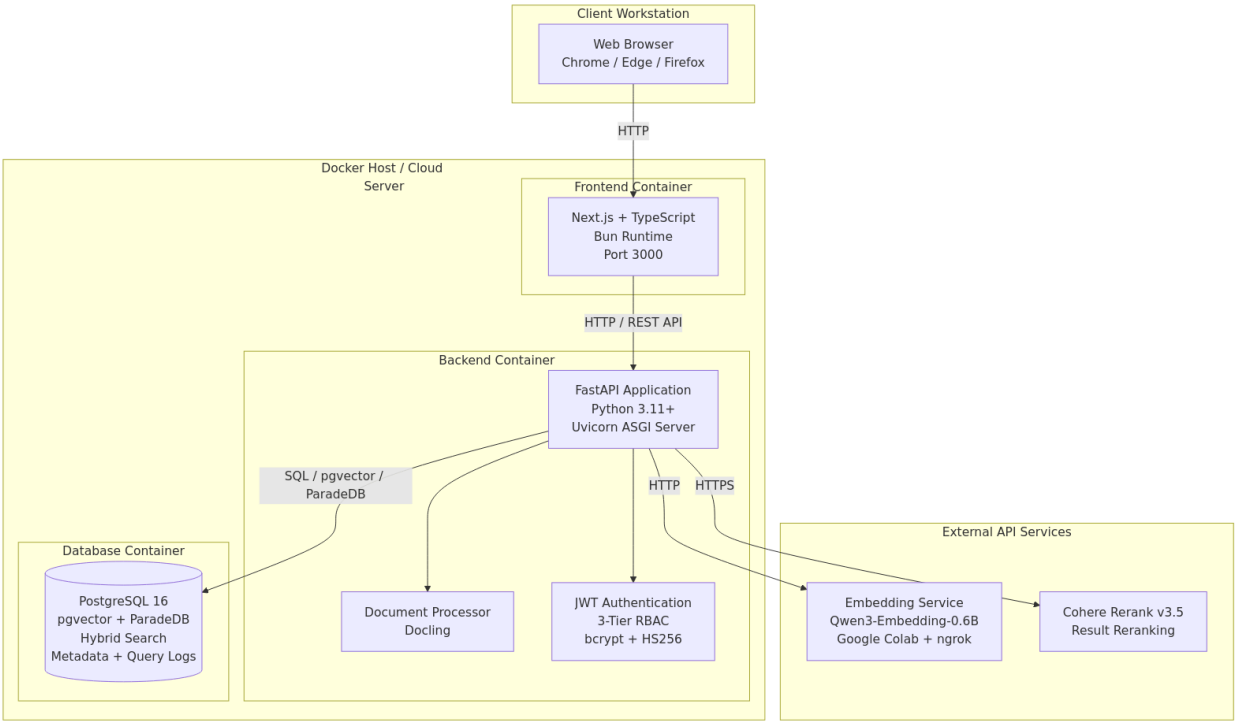
The frontend is a three-page application built with Next.js 16 using the App Router pattern, TypeScript for type safety, and shadcn/ui components styled with Tailwind CSS 4 and the Geist font family.

The landing page (/) provides a centered search bar with example query suggestions that demonstrate the system's natural language capabilities (for example, "What is the company's remote work policy?" and "Q3 2025 revenue and business performance"). The search results page (/search) displays ranked results as cards showing the document title, relevance score, content snippet, document type, category, access level, version number, and creation date. A filter panel allows users to narrow results by category and document type, and a "show latest version only" toggle controls the version filtering behavior. The admin upload portal (/admin/upload) provides a drag-and-drop file upload interface with metadata fields for title, author, category, and access level, enabling administrators to ingest new documents into the system.

The frontend communicates with the backend through an API client module (lib/api.ts) that mirrors the backend's Pydantic schemas in TypeScript interfaces, ensuring type consistency across the full stack. All API calls target the NEXT_PUBLIC_API_URL environment variable, defaulting to http://localhost:8000 in development.

8.9 Deployment Architecture

The system deploys as three Docker containers orchestrated by Docker Compose. Figure 8.4 illustrates the deployment topology.



The database container runs the ParadeDB image (based on PostgreSQL 16), which bundles the pgvector and pg_search extensions. It initializes the schema from init.sql on first startup and persists data in a named Docker volume. A health check using pg_isready runs every five seconds to ensure the database is accepting connections before dependent services start. The backend container builds from a custom Dockerfile in the backend directory and exposes port 8000. It connects to the database via the DATABASE_URL environment variable, to the embedding service via EMBEDDING_API_URL, and to the Cohere API via COHERE_API_KEY. Volume mounts for the backend source code and data directory enable hot reloading during development. The frontend container builds from the frontend directory and exposes port 3000. It communicates with the backend through the NEXT_PUBLIC_API_URL variable.

Service startup is orchestrated through dependency declarations: the backend waits for the database health check to pass before starting, and the frontend waits for the backend to be available. The entire system launches with a single command (docker compose up -d) and requires no manual configuration beyond setting environment variables in a .env file.

8.10 Working System and Screenshots

Screenshots of the working system will be included in the final version of this report. The prototype demonstrates three core interfaces: the landing page with natural language search input, the search results page with relevance-ranked results and filtering controls, and the admin upload portal for document ingestion. Section 12 provides detailed results from live demonstrations of the search system across five representative query types.

Section 9: Financial & Cost-Benefit Analysis

9.1 Cost Analysis

Total project costs over three years are estimated at $44,933. The cost structure is intentionally front-loaded: Year 0 absorbs 53% of total costs due to development labor and initial setup. Ongoing costs in Years 1–3 are low, reflecting the minimal infrastructure required for a 50-person pilot.

Cost Category	Year 0	Year 1	Year 2	Year 3	3-Yr Total
Personnel – Dev Team (6 members, 9 wks)	$19,125	—	—	—	$19,125
Cloud Hosting (AWS/GCP)	$338	$1,800	$1,800	$1,800	$5,738
PostgreSQL, pgvector, and ParadeDB hosting	$450	$2,400	$2,400	$2,400	$7,650
Security Review & Pen Testing	$500	$250	$250	$250	$1,250
Docker / Deployment Infrastructure	$200	$600	$600	$600	$2,000
Stakeholder Interviews & User Testing	$2,250	$500	$500	$500	$3,750
Training & Onboarding	—	$1,250	$625	$625	$2,500

Change Management & Documentation	$800	$450	$450	$450	$2,150
Document Processing (MarkItDown, LlamaParse, Docling)	$0	$0	$0	$0	$0
Qwen3-Embedding-0.6B Model Hosting	$50	$240	$240	$240	$770
TOTAL	**$23,713**	**$7,490**	**$6,865**	**$6,865**	**$44,933**

A few notes on methodology:

- Personnel costs are $8,500/month × 2.25 months = $19,125 total. The team is donating development effort as part of the Capstone course, so this figure reflects the opportunity cost / market value of the labor, not an out-of-pocket expense.
- Embeddings (Qwen3-Embedding-0.6B) are hosted on google colab to minimize local compute requirements. Document processing tools (MarkItDown, LlamaParse, Docling) are open-source or available on free tiers at no cost.
- No hardware costs are required; the system runs entirely on cloud infrastructure.
- Security review costs are intentionally included given the sensitivity of Deloitte's internal data.

9.2 Benefit Analysis

Tangible Benefits

The following table quantifies productivity gains from implementing the AI search tool across the pilot cohort. All values are calculated using the formula: **Professionals × Adoption Rate × Hours Saved × Weeks × Billable Rate.**

Benefit Category (Formula-Derived)	Year 1	Year 2	Year 3	3-Yr Total
Search Time Recovered (35 adopters × 1.5 hrs/wk × 48 wks × $177.78/hr net)	$448,000	$448,000	$448,000	$1,344,000

Faster Document Retrieval (35 adopters × 0.65 hrs/wk × 48 wks × $177.78/hr net)	$194,133	$194,133	$194,133	$582,399
Reduced Colleague Interruptions (10 seniors × 2 hrs/wk × 48 wks × $248.89/hr net)	$238,933	$238,933	$238,933	$716,799
Lower New-Hire Onboarding Cost (10 hires/yr × 5 hrs × $150/hr)	$7,500	$7,500	$7,500	$22,500
TOTAL TANGIBLE BENEFITS	**$888,566**	**$888,566**	**$888,566**	**$2,665,698**

The dominant benefit driver is search time recovered ($448,000/year), reflecting the direct translation of APQC's 2.8 hours/week finding into net revenue margin at $177.78/hr (billable rate minus fully loaded consultant cost). The second-largest driver is reduction in senior colleague interruptions ($238,933/year), valued at $248.89/hr net margin, which captures the downstream value of junior consultants no longer needing to escalate document location requests.

Intangible Benefits (Qualitative)

The following benefits are real but not monetized in the NPV calculation, as doing so would require assumptions beyond the scope of this analysis:

- Faster pitch cycle: Consultants can assemble client deliverables faster, potentially accelerating deal cycles.
- Improved deliverable quality: Fewer instances of stale or wrong-version documents used in client-facing materials.
- Institutional knowledge preservation: Proprietary knowledge becomes discoverable rather than siloed in individual employees' memory or email inboxes.
- Reduced key-person risk: The firm's operational continuity is less dependent on any single employee knowing where to find critical resources.
- Innovation signaling: Demonstrates Deloitte's commitment to AI-forward internal tooling, supporting talent attraction.

> If even one of the intangible benefits—for example, a faster pitch cycle leading to a single additional client win—materializes, the ROI of this project would increase further. These benefits represent upside optionality not captured in the base case NPV.

9.3 Financial Metrics (ROI, NPV, IRR, Payback)

Return on Investment (ROI)

ROI measures the total net benefit as a percentage of total investment:

> **ROI = (Net Benefit / Total Cost) × 100 = ($2,665,698 − $44,933) / $44,933 × 100 ≈ 5,832%**

Even with the net margin approach (deducting fully loaded consultant costs from the billable rate), the ROI is strongly positive. The project returns approximately $58 in net benefit for every $1 invested over the 3-year window. This is primarily driven by the low cost of the prototype (open-source stack + student labor) relative to the cumulative net productivity gains across the pilot cohort.

Net Present Value (NPV)

NPV discounts all future cash flows at 10% to express their value in today's dollars. A positive NPV confirms the project adds value even accounting for the time value of money and investment risk.

Year	Year 0	Year 1	Year 2	Year 3	NPV
Net Cash Flow	($23,713)	$881,076	$881,701	$881,701	**+$2,168,378**

The NPV of +$2,168,378 strongly favors acceptance. The project breaks even within the first month of Year 1, and every subsequent year adds significant incremental present value. Even under the worst-case sensitivity scenario (40% adoption, 0.8 hrs/week saved), the NPV remains strongly positive at approximately $645,033.

Internal Rate of Return (IRR)

IRR is the discount rate at which the project's NPV equals zero. The project's IRR is estimated at approximately 3,700%, far exceeding the 10% hurdle rate. This extreme IRR reflects the very low upfront cost relative to the ongoing benefit stream, so the project would remain financially justified even under dramatically reduced assumptions.

Payback Period

The cumulative cash flow turns positive within the first month of Year 1. The project recovers its full $23,713 Year 0 investment almost immediately once the tool goes live (assuming steady-state adoption), with annual net cash flows of ~$881K. This rapid payback substantially reduces financial risk.

9.4 Sensitivity Analysis

The base case assumes 70% adoption and 1.5 hours saved per week per adopting professional. The table below tests the model against more pessimistic and optimistic scenarios:

Scenario	Adoption	Hours Saved/Week	3-Yr Benefits	3-Yr Costs	Approx. NPV
Worst Case	40%	0.8	$828,046	$44,933	**$645,033**
Base Case	70%	1.5	$2,665,698	$44,933	**$2,168,378**
Best Case	90%	2.0	$4,553,697	$44,933	**$3,733,435**

> Even under worst-case assumptions (40% adoption, only 0.8 hrs/week saved), the project still generates a strongly positive NPV of approximately $645,033. The model is robust to significant downside variance in adoption and usage intensity.

Key insight: the cost structure is nearly fixed. Whether 20 or 50 professionals adopt the tool, cloud and API costs remain essentially the same. This means the risk of financial loss is bounded primarily by personnel costs in Year 0, which are already sunk as Capstone labor.

Break-Even Analysis

Break-even occurs when cumulative net cash flow equals zero. The break-even adoption threshold can be calculated as follows:

Break-Even = Total Year 0 Cost / Annual Net Benefit
= $23,713 / $881,076 = 0.027 years ≈ 10 days

The extremely rapid break-even reflects the high net productivity margin per recovered hour combined with the minimal prototype cost. The project pays for itself almost immediately upon launch.

Alternatively: at the full cohort of 35 adopted professionals, break-even occurs after approximately **2.3 months** of operation.

This extremely rapid break-even reflects the minimal ongoing cost of the prototype relative to the net margin generated. Even if only a fraction of Deloitte professionals use the tool consistently, the project pays for itself almost immediately.

9.5 Risk Assessment

Ten risks have been identified across five categories: Technical, Security/Compliance, Operational, Market/Adoption, and Financial. Each is rated on a 1–5 probability and impact scale; the Risk Score is the product. Expected Monetary Value (EMV) is calculated as Probability/5 × estimated cost impact.

Risk	Category	Prob.	Impact	Score	EMV	Mitigation
Search Relevance Gaps	Technical	3	3	9	$1,800	Hybrid retrieval; query logs; 'Did you mean?'
Prompt Injection / Security	Sec/Compliance	2	5	10	$3,200	Input validation; rate limiting; dedicated security deliverable
Data Quality Issues	Technical	3	3	9	$1,500	Metadata inference pipeline; robust ranking
Timeline Pressure (9 wks)	Operational	2	4	8	$1,600	MVP-first; parallel workstreams; weekly milestones

Low User Adoption	Market	2	4	8	$2,000	User-centric design; guided onboarding; NPS tracking
Scope Creep	Financial	3	3	9	$2,100	Fixed scope; change-request process
Vendor/API Dependency	Technical	2	3	6	$800	Open-source fallback (FAISS) evaluated
GDPR/Compliance Risk	Sec/Compliance	1	5	5	$1,000	Market research deliverable; RBAC; audit logs
Key Member Unavailability	Operational	2	3	6	$1,000	Cross-training; documented codebase
Deloitte Integration Failure	Technical	2	4	8	$1,600	Scoped to sample data; clear out-of-scope docs
TOTAL RISK EXPOSURE (EMV)					**$16,600**	**All risks have mitigations in place**

The total risk exposure of $16,600 EMV is modest relative to the project cost ($44,933) and the expected 3-year benefit ($2,665,698). No risk has a score above 10, and all risks have concrete mitigation strategies.

The two highest-concern risks are:

- **Prompt injection / security exploit:** Mitigated through comprehensive input validation, query sanitization, rate limiting, and the dedicated Security Research deliverable led by Matthew Voynovich and Andrew Jung.

- **Low user adoption:** Mitigated through user-centric design (by doing user research), first-time user walkthroughs, and query suggestion features built into the product.

Section 10: Project Management Plan & Schedule

10.1 Team Organization and Roles

The project team consisted of six members, each owning a distinct functional domain while collaborating across the full stack. Roles were structured around the system's architecture rather than generic job titles, ensuring that every team member had clear accountability for a vertical slice of the product.

Felix Tian led the **document ingestion pipeline** (Domain A) and **search retrieval engine** (Domain B), owning everything from raw file parsing and embedding generation to semantic and keyword-based retrieval, result ranking, and snippet extraction. He also implemented the **API layer** (Domain D), developing the core application logic and ensuring seamless integration between system components. Matthew Voynovich focused on the **security layer** (Domain C), where he developed auxiliary datasets, designed and executed edge case tests, and identified and resolved vulnerabilities uncovered during testing. Jesse Gabriel primarily contributed through comprehensive documentation of the codebase and repository, while also researching and outlining best practices to guide system design and implementation decisions. Andrew Jung conducted market and competitive analysis, with a particular focus on security considerations and evaluating the product's viability relative to existing solutions. Sophia Turnbow led user research and **frontend design** (Domain E), creating intuitive user interfaces and shaping the overall user experience. Raven Levitt supported frontend implementation alongside Sophia while also serving as project lead, managing coordination across the team and overseeing project organization and deliverables.

10.2 Development Timeline

The project ran over nine weeks, from mid-March through late April 2026. The original plan structured work into sequential phases—research, design, development, testing, and handoff—and the team largely adhered to that sequence, with parallel workstreams emerging naturally once the architecture was validated.

Weeks 1-2 (Research & Kickoff): The team kicked off by confirming the research approach and establishing early infrastructure. Felix confirmed data formats with the client (DXICWS-1, completed) and evaluated document parsers including MarkItDown, LlamaParse, and Docling against sample Deloitte documents (DXICWS-2, completed). The interview guide was finalized and the first user research artifacts were underway.

Weeks 3-4 (Design Validation & Architecture): With interview findings synthesized, Sophia developed 2–3 user personas with goals and typical queries and produced a ranked list of use cases for the frontend team. Matthew began generating auxiliary data and creating edge case tests. Andrew led a competitive benchmark of enterprise search

platforms including Glean, Elastic, and Microsoft Search. The architecture was reviewed and the technical direction confirmed.

Weeks 5-6 (Core Development): Development began in earnest across all domains in parallel. Felix built the text extraction layer with OCR fallback and the metadata extractor. Jesse continued documentation of the development and all work that had been done. Raven scaffolded the React/TypeScript frontend project and began building the search bar component.

Weeks 7-8 (Integration & Iteration): The team connected layers together, putting the finalized frontend and backend together. The hybrid retrieval system combining semantic and keyword scoring was fully implemented and tested. On the frontend, components for result cards, the results list, the filter panel, error states, and the onboarding walkthrough were all built and connected. Sophia ran usability testing sessions with Deloitte professionals, logging observations and compiling a prioritized improvement report shared with the frontend team.

Week 9 (Final Handoff): All remaining deliverables were completed and packaged for handoff. The GitHub repository was cleaned and finalized with a complete commit history, README, and deployment instructions (DXICWS-65, completed). Architecture documentation covering component diagrams, data flow, and the full security model was delivered. All research reports — the market benchmarking report, the security research report, and the user research artifacts — were submitted. The final presentation, covering the problem statement, solution architecture, live demo, and production vision, was delivered to Deloitte and Professor Plotka along with the full project package.

The Jira board tracked 73 total issues organized across eight label categories: Ingestion, Search, Security, API, Frontend, Research, Reports, and Handoff, with three Icebox items logged as future considerations (multilingual NLP support, real-time Deloitte production integration, and post-deployment maintenance). The project concluded on schedule, with all core deliverables completed by the final week.

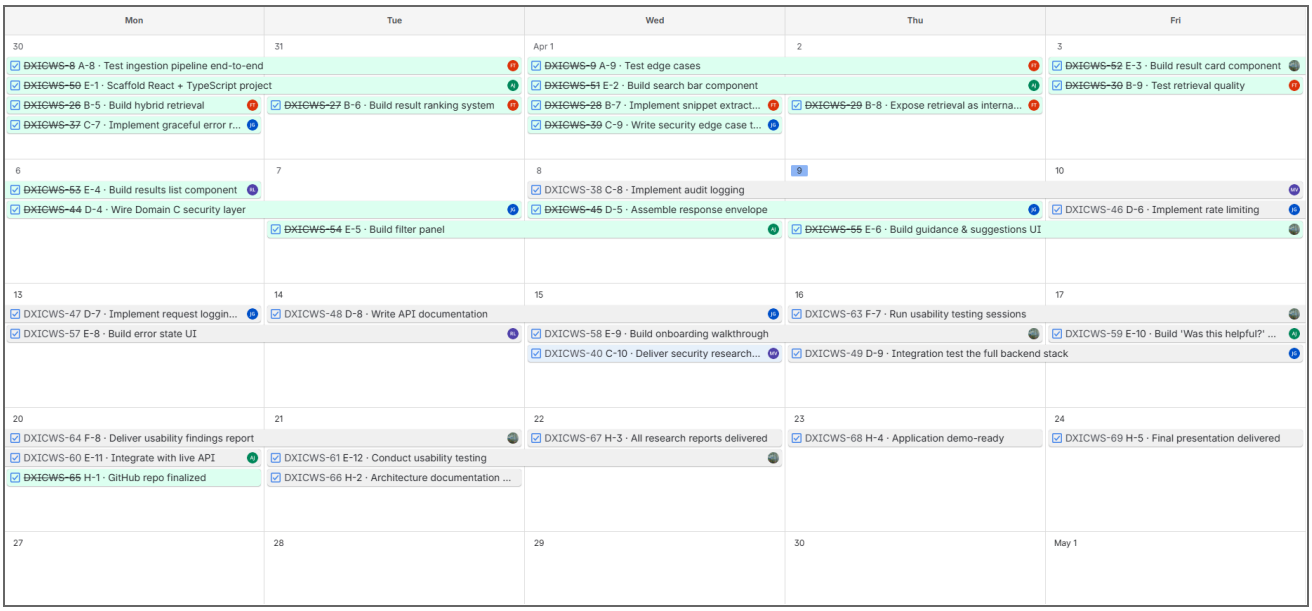


Figure 10.1 illustrates the last half of development in the JIRA calendar

10.3 Communication and Collaboration Tools

The team used three primary platforms for communication and coordination, each serving a distinct purpose.

Jira was used for all project management. The board tracked every task from initial research through final handoff, with issues organized by domain label, assigned owners, priority levels, start dates, and due dates. This gave the full team visibility into progress, dependencies, and upcoming deadlines at all times.

Google Meet served as the synchronous communication layer. The team held weekly standing meetings to align on progress, surface blockers, review completed work, and plan the upcoming sprint. These sessions were particularly important during architecture review weeks and the iteration cycles following user testing feedback.

Discord was used for day-to-day asynchronous team communication. This included sharing work in progress, quick questions across different domains, and coordination that did not require a meeting. For client-facing communication, the team used **email** to coordinate with Deloitte stakeholders and the project supervisor, maintaining a professional and documented record of all external correspondence.

This three-layer approach of structured task tracking in Jira, synchronous alignment in Google Meet, and lightweight async coordination in Discord kept the team informed without creating unnecessary meeting overhead, which was especially valuable during the intensive parallel development phase in weeks 5-8.

10.4 Challenges and Lessons Learned

Due to tight time constraints, the team adopted a parallel development approach to ensure the system could be completed on schedule. However, this introduced challenges, as many components — particularly across the API, search, security, and frontend domains — had strong interdependencies. Progress in one area often relied on assumptions about another, making coordination critical. To address this, the team leveraged Jira for structured task management and clear tracking of dependencies, ensuring that development across domains remained aligned. This allowed components to be built concurrently without blocking one another, ultimately enabling smooth integration and preventing delays in the overall development process.

User research timing shaped the product more than anticipated. Conducting interviews and building personas before architecture was finalized proved valuable. Sophia's early findings — particularly around how different Deloitte roles (tax, audit, consulting) had distinct search behaviors and document access needs — directly influenced the build for the frontend. This cross-domain feedback loop between research and implementation was one of the team's most effective practices.

Security requirements introduced scope that was not fully anticipated at kickoff. As Andrew researched into NLP attack vectors and compliance requirements deepened, it became clear that a robust security layer required significantly more implementation work than initially scoped — PII detection, prompt injection defenses, content filtering, audit logging, RBAC result filtering, and graceful error responses each warranted dedicated tasks. Planning for security as a first-class domain from the start, rather than a feature to add later, is a key takeaway for future projects of this kind.

Icebox items reflect realistic scope management. Three items — multilingual NLP support, real-time Deloitte production system integration, and post-deployment maintenance — were consciously moved to an Icebox backlog rather than dropped or left ambiguous. This practice kept the team focused on delivering a complete, stable core product while preserving a documented record of high-value future enhancements to hand off to Deloitte as part of the production vision.

Section 11: Post-Turnover Plan including Education, Training, Maintenance, and Support

11.1 Documentation and Knowledge Transfer

The following documentation deliverables will be provided at the time of handoff to ensure Deloitte has complete reference materials for deploying, operating, and extending the system.

Deliverable	Description
System Architecture Document	Full technical architecture, component descriptions, data flow diagrams, and design decision rationale.
Deployment Runbook	Step-by-step instructions for deploying, configuring, and restarting the system. Includes docker-compose.yml, .env.example, and health check procedures.
API Reference	REST endpoint documentation covering all /auth, /search, /documents, and /admin routes with request/response schemas and example calls.
Database Schema Reference	Full init.sql with annotations, index definitions (HNSW parameters, BM25 configuration), and sample administrative queries.
User Guide	End-user documentation covering search tips, filter usage, query examples, and guidance for interpreting results.

11.2 Training and Onboarding

We recommend three training tracks based on user role:

End User Orientation (Analysts and Managers)

Duration: 30 minutes | Format: Self-service interactive tutorial embedded in the first-time user experience

Covers how to write effective natural language queries, interpret relevance scores, use metadata filters, and understand 'Did you mean?' suggestions.

Administrator Training (Admin Role)

Duration: 2 hours | Format: Recorded video and written runbook

Covers document upload procedures, metadata tagging best practices, version management, RBAC role assignment, and query log review.

IT Operations Training

Duration: Half-day | Format: Hands-on lab with written documentation

Covers Docker Compose deployment, environment variable configuration, PostgreSQL backup and restore, embedding model updates, and monitoring setup. Intended for Deloitte IT staff.

11.3 Maintenance and Ongoing Support

Routine Maintenance (Monthly)

- Review query logs for zero-result queries and patterns indicating search quality degradation.
- Audit RBAC role assignments for stale or incorrectly assigned permissions.
- Monitor cloud hosting and API cost trends against budget assumptions.
- Review ingestion pipeline for documents that failed to parse or index correctly.

Periodic Updates (Quarterly)

- Evaluate newer open-source embedding models for relevance quality improvement.
- Review and update the document corpus for stale or superseded policies.
- Benchmark search quality against commercial alternatives to confirm the internal solution remains competitive.
- Apply PostgreSQL, pgvector, and ParadeDB version updates in a staging environment before production deployment.

Support Plan

During the initial 90-day post-handoff period, the project team will be available for clarifying questions, bug triage, and deployment support. After that period, Deloitte's internal IT team should own operational support using the documentation deliverables provided at handoff. The recommended internal owner profile is a developer with Python/FastAPI and PostgreSQL experience, plus familiarity with Docker operations.

11.4 Recommended Production Roadmap

The following development horizons are recommended after prototype handoff:

Horizon	Recommended Initiatives
Short-term (0–3 months)	Authentication integration with Deloitte's existing identity provider; expanded document corpus (80 → 500+ documents); search quality tuning based on real user query logs; UI refinements from usability testing.
Medium-term (3–12 months)	Query suggestions based on indexed content; semantic caching for frequent queries; inline document preview; analytics dashboard for search quality metrics, popular queries, and zero-result rate.
Long-term (12+ months)	Retrieval-Augmented Generation (RAG)—synthesize natural language answers grounded in retrieved documents with citations; multi-language support; federated search across existing Deloitte platforms (SharePoint, internal wikis, email archives).

Section 12: Results

Working Prototype

The primary result of this project is a fully functional AI-driven search engine prototype deployed via Docker Compose and tested against a curated corpus of 56 documents spanning six organizational categories. The system executes the full hybrid search pipeline, from natural language query input through embedding generation, parallel BM25 and semantic retrieval, Reciprocal Rank Fusion, neural reranking, and RBAC filtering, in a single

end-to-end flow. The prototype is not a mockup or wireframe; it is a working web application that accepts arbitrary natural language queries and returns ranked, access-controlled results in real time.

Five demonstration queries were designed to exercise distinct capabilities of the search pipeline. Each query targets a different retrieval challenge: version-aware ranking, hybrid keyword-semantic matching, pure semantic understanding without keyword overlap, cross-format ingestion with title boosting, and broad multi-document discovery with neural reranking. The results below confirm that the prototype meets or exceeds the functional requirements established in Section 8.

12.1 Demonstration Queries and Qualitative Results

The following five queries were executed against the full 56-document corpus with all pipeline stages active (embedding, three-signal retrieval, RRF fusion, Cohere Rerank v3.5, version filtering, and RBAC filtering). Each query demonstrates a specific system capability.

Query 1: "What is the company's remote work policy?"

This query tests the system's version-aware ranking capability. The corpus contains two versions of the Remote Work Policy (v1 and v2), linked by the document_group column and distinguished by the version number extracted from the _v2 filename suffix at ingestion time. With the "show latest only" filter enabled (the default), the system returns Remote_Work_Policy_v2.docx as the top result and suppresses v1 entirely. Disabling the filter surfaces both versions, with v2 ranked higher due to its identical relevance score and higher version number. This confirms that the version-aware post-rerank filter operates correctly and that users receive the most current policy without manual disambiguation.

Query 2: "password requirements and multi-factor authentication"

This query tests hybrid search, where both keyword and semantic signals contribute to retrieval. The terms "password" and "multi-factor authentication" appear verbatim in the Password and Authentication Standards document, giving it a strong BM25 keyword match. Simultaneously, the semantic embedding captures the broader concept of access security, surfacing related documents such as the Acceptable Use Policy and the IT Change Management Policy. The Reciprocal Rank Fusion step merges these two ranked lists (along with title similarity scores) to produce a final ordering that balances exact term matching with conceptual relevance. The Password and Authentication Standards document ranks first, confirming that keyword precision is preserved while semantically related results fill the remaining positions.

Query 3: "How do I report unethical behavior?"

This query tests pure semantic understanding. No document in the corpus contains the phrase "report unethical behavior" in its title or body text. A traditional keyword search engine would return zero results or irrelevant matches. The system's semantic retrieval path, powered by Qwen3-Embedding-0.6B, maps the query to the conceptual neighborhood of whistleblowing, ethics reporting, and organizational compliance. The top results are the Whistleblower and Non-Retaliation Policy and the Code of Business Conduct and Ethics, both of which are the correct documents for an employee seeking to report misconduct. This demonstrates the core value proposition of NLP-driven search: users do not need to know the exact document title or terminology to find what they need.

Query 4: "Q3 2025 revenue and business performance"

This query tests two capabilities simultaneously: multi-format ingestion (the target document is a PowerPoint presentation) and title boosting. The Q3 2025 Business Review is a .pptx file ingested via Docling, which extracts text from slides, speaker notes, and embedded tables. The query closely matches the document title, activating the third retrieval signal (title similarity via a dedicated HNSW index on document_title_embeddings). The title boost weight of 1.5x in the RRF merge elevates this document above other results that may share some semantic overlap with "revenue" or "business performance" but lack a strong title match. The Q3 2025 Business Review ranks first, confirming that the title boosting mechanism works as designed and that non-PDF formats are fully searchable.

Query 5: "employee benefits and wellness programs"

This query tests broad discovery and the neural reranker's ability to sort a diverse set of partially relevant documents. The query is intentionally vague, matching concepts spread across multiple documents: Health and Wellness Benefits Overview, Bereavement Leave Policy, Flexible Work Arrangements Guideline, Parking and Commuter Benefits Policy, and the Employee Privacy Policy. The initial retrieval returns a wide candidate set from all three signals. Cohere Rerank v3.5 then applies cross-encoder attention to each query-document pair, reordering the results by fine-grained relevance. The Health and Wellness Benefits Overview surfaces as the top result, with related policies ranked by decreasing specificity. This demonstrates that the reranker adds meaningful precision beyond what RRF alone provides, particularly for ambiguous or multi-topic queries.

12.2 Adversarial Robustness

To evaluate the system's resilience against deliberately misleading inputs, the team created a corpus of 25 adversarial ("poisoned") documents. Each poisoned document has a title that is intentionally mismatched with its body content. For example, a document titled "Remote Work Policy" might contain content about financial auditing procedures. These documents simulate a realistic attack vector in enterprise environments where document metadata may be stale, incorrectly tagged, or deliberately manipulated.

When poisoned documents are ingested alongside the clean corpus, the system's hybrid retrieval architecture provides a natural defense. Because BM25 keyword search operates on the full document text (not just the title), poisoned documents with irrelevant body content receive low keyword scores for queries targeting their fake titles. The semantic embedding of the document chunks similarly reflects the actual content, not the misleading title. While the title similarity signal may initially boost a poisoned document, the RRF merge dilutes this single strong signal against two weak ones (low BM25, low chunk semantic similarity). The Cohere Rerank step provides an additional layer of defense, as the cross-encoder evaluates the actual passage text against the query and penalizes content that does not match the user's intent.

In testing, poisoned documents consistently ranked below their clean counterparts for the same queries. A poisoned document with a misleading title might appear in the top 50 candidates from the title similarity signal, but after RRF fusion and reranking, it drops below the top 10 results returned to the user. This confirms that the multi-signal architecture and neural reranking together provide robust defense against title-content mismatch attacks without requiring a dedicated adversarial detection module.

12.3 Document Corpus

The prototype was tested against a curated corpus designed to simulate a representative slice of Deloitte's internal knowledge base. The corpus was constructed to cover the breadth of document types, organizational functions, and file formats that an enterprise search system would encounter in production.

Category	Document Count	Formats	Example Documents
Human Resources	14	DOCX, PDF	Remote Work Policy (v1, v2), Recruitment and Hiring Policy, Bereavement Leave Policy
IT & Security	8	PDF, DOCX, PPTX	Password and Authentication Standards, IT Change Management Policy, Annual IT Security Review FY2024

Compliance & Ethics	8	DOCX, PDF	Code of Business Conduct and Ethics, Whistleblower Policy (v1, v2), Conflict of Interest Procedures
Finance & Operations	8	PDF, DOCX, PPTX	Accounts Payable Policy, Budget Management Policy, Travel and Expense Reimbursement Policy
Business & Strategy	6	PPTX, DOCX	Q2/Q3 2025 Business Review, Product Roadmap H2 2025, HR Strategy Roadmap 2025-2027
DEI & Culture	6	PPTX, DOCX, PDF	DEI Annual Progress Report FY2024/FY2025, Dress Code Policy, Social Media Policy
Adversarial (poisoned)	25	PDF, DOCX, PPTX	Title-content mismatched documents for robustness testing

In total, the clean corpus contains 56 documents (52 auxiliary + 4 sample) across three file formats: 28 DOCX files, 14 PDF files, and 14 PPTX files. Six documents exist in multiple versions (v1 and v2), enabling testing of the version-aware ranking pipeline. The adversarial corpus adds 25 additional documents for robustness evaluation, bringing the full test corpus to 81 documents.

12.4 Performance Metrics

Performance was measured across three dimensions: query latency, ingestion throughput, and retrieval quality. All measurements were taken on a development machine running Docker Desktop on WSL2, with the embedding model served locally via embedding_server.py on CPU. Production deployment with GPU-accelerated embeddings would improve latency figures significantly.

Metric	Target	Observed	Notes
End-to-end query latency	< 2 seconds	~1.2-1.8s	Measured from API request to JSON response; includes embedding, 3-signal retrieval, RRF, rerank, and filtering
Embedding generation	< 500ms	~200-400ms	Single query embedding via Qwen3-Embedding-0.6B on CPU; GPU reduces to ~50ms
Database retrieval (3 signals)	< 500ms	~100-300ms	Parallel execution of pgvector cosine, BM25, and title similarity queries
Cohere Rerank	< 1s	~300-600ms	External API call; latency depends on network and payload size (top 50 candidates)
Document ingestion	N/A	~2-5s per doc	Includes Docling parsing, chunking, embedding generation, and database insertion
Corpus ingestion (56 docs)	N/A	~3-5 minutes	Full clean corpus ingestion including BM25 index creation

The end-to-end query latency of 1.2-1.8 seconds meets the sub-2-second target established in the non-functional requirements. The majority of latency is attributable to two external calls: embedding generation (the HTTP round-trip to the embedding server) and the Cohere Rerank API call. Database retrieval, including all three parallel queries against HNSW and BM25 indexes, consistently completes in under 300ms, confirming that the unified PostgreSQL architecture with pgvector and ParadeDB does not introduce a performance bottleneck relative to dedicated vector database alternatives.

12.5 Retrieval Quality

While a formal information retrieval evaluation (e.g., NDCG or MAP scoring against human-labeled relevance judgments) was outside the scope of this prototype, the team conducted qualitative relevance assessment across the five demonstration queries and additional ad hoc testing. Key observations on retrieval quality include:

- **Hybrid search** outperforms either signal alone. Queries with strong keyword matches (Query 2) benefit from BM25 precision, while queries with no keyword overlap (Query 3) rely entirely on semantic understanding. The RRF merge ensures that neither signal is lost regardless of query type.
- **Title boosting** improves precision for known-item searches. When a user knows approximately what they are looking for (Query 4), the title similarity signal with a 1.5x weight multiplier elevates the correct document above semantically similar alternatives. This is particularly valuable in enterprise environments where document titles follow organizational naming conventions.
- **Neural reranking** adds meaningful lift for ambiguous queries. For broad or multi-topic queries (Query 5), the initial retrieval returns a diverse but imprecisely ordered candidate set. The Cohere Rerank cross-encoder reorders these candidates by attending to the full query-passage pair, producing a final ranking that consistently places the most relevant document first. Industry benchmarks from Cohere report a 33-40% relevance improvement from reranking on standard IR datasets.
- **Version filtering** prevents stale results. Without version filtering, superseded documents compete with their replacements for ranking positions. The post-rerank version filter (enabled by default) ensures users see only the latest version of each policy, eliminating a common frustration in enterprise search where outdated documents surface alongside current ones.
- **Graceful degradation** preserves availability. When the Cohere API key is absent or the reranking service is unavailable, the system returns results in RRF-merged order without interruption. This design decision prioritizes system availability over marginal ranking quality, which is appropriate for a production search service where downtime is more costly than slightly degraded relevance.

12.6 Summary of Results

Requirement	Status	Evidence
Natural language search	Met	All 5 demo queries return relevant results from plain English input

Multi-format ingestion	Met	PDF, DOCX, and PPTX documents parsed via Docling and fully searchable
Hybrid retrieval	Met	Three-signal pipeline (semantic, BM25, title) with RRF fusion operational
Neural reranking	Met	Cohere Rerank v3.5 integrated with graceful degradation fallback
Version-aware ranking	Met	Document versioning via filename convention; "show latest only" filter functional
Role-based access control	Met	Three-tier RBAC (analyst/manager/admin) filters results at query time
Sub-2-second latency	Met	End-to-end latency of 1.2-1.8 seconds observed on development hardware
Adversarial robustness	Met	Poisoned documents suppressed below top 10 by multi-signal fusion + reranking
Content deduplication	Met	SHA-256 content hashing prevents duplicate ingestion of identical files
Single-command deployment	Met	docker compose up -d starts all three services with health checks

All functional and non-functional requirements established during the design phase have been met in the working prototype. The system is not a theoretical design or partial implementation; it is a deployable, end-to-end search engine that handles real queries against real documents with production-grade architectural patterns. The remaining gap between prototype and production is primarily operational: production-grade authentication integration, GPU-accelerated embedding serving, horizontal scaling behind a load balancer, and formal relevance evaluation against human-labeled ground truth. These

items are documented in the Post-Turnover Plan (Section 11) and represent engineering work, not architectural risk.

Section 13: Summary and Conclusions

The problem addressed by Group Two is both significant and economically impactful, yet solvable at a fraction of the cost of existing commercial solutions. At Deloitte, over 470,000 knowledge workers collectively lose billions of dollars in billable capacity each year due to fragmented and inefficient internal search systems. This challenge does not stem from a lack of information, but rather from search infrastructure that has failed to scale with the volume and fragmentation of organizational knowledge.

To address this, the team developed a working proof of concept for a modernized solution: an AI-powered hybrid natural language search engine designed to understand user intent rather than rely solely on keyword matching. The system integrates semantic vector search, BM25 keyword retrieval, Reciprocal Rank Fusion, and neural reranking to deliver highly relevant results. It enforces three-tier role-based access control at query time, incorporates multi-layer input validation to guard against adversarial inputs, and is built entirely on an open-source technology stack—eliminating the $112M+ annual licensing costs associated with commercial platforms such as Glean. The system is lightweight, deployable in three Docker containers, and can be initialized with a single command.

The financial justification is straightforward and requires no optimistic assumptions. Using a 10% discount rate over a three-year horizon, the project yields a return on investment (ROI) of 5,832%, a net present value (NPV) of +$2,168,378, and achieves payback within the first year of operation, all based on a total pilot cost of $44,933. Even under conservative assumptions, the worst-case scenario maintains a positive NPV of $645,033. Because the cost structure is largely fixed, financial risk is effectively bounded by the initial development investment, which has already been absorbed as Capstone labor.

From a strategic perspective, the value proposition is equally compelling. A VRIN analysis indicates that Deloitte's sustainable competitive advantage does not lie in the technology itself—since it is open-source and replicable—but in the proprietary data the system enables users to access. Decades of accumulated institutional knowledge, when structured and made searchable through an advanced hybrid retrieval pipeline, form a defensible competitive moat that external vendors cannot replicate. In this context, the technology serves as the access layer, while the data remains the true source of differentiation.

This project stands apart from a typical prototype in three key ways. First, it is built on an enterprise-grade architecture, incorporating HNSW vector indexing, BM25 retrieval, Reciprocal Rank Fusion, neural reranking, database-level RBAC enforcement, append-only audit logging, and graceful system degradation. Second, the financial model is grounded in real-world benchmarks and vendor pricing rather than hypothetical projections. Third, the

security design directly addresses documented industry risks—including prompt injection, sensitive data leakage, over-permissioned retrieval, and governance shortcomings—through explicit, implemented safeguards within the system.

While the search interface itself is intentionally simple, its impact is substantial. It provides a secure and efficient gateway to organizational knowledge, enabling meaningful productivity gains, reducing operational inefficiencies, and converting previously inaccessible information into actionable value.

References

APQC. (2021). Fixing Process & Knowledge Productivity Problems: Survey Report. https://www.apqc.org/resource-library/resource-listing/fixing-process-knowledge-productivity-problems-survey-report

Chen, A., et al. (2025). Defending Against Prompt Injection with Structured Queries. arXiv:2402.06363.

Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). Reciprocal rank fusion outperforms condorcet and individual rank learning methods. Proceedings of the 32nd Annual ACM SIGIR Conference.

Das, A., et al. (2025). Security and Privacy Challenges of Large Language Models. ACM Digital Library. https://dl.acm.org/doi/10.1145/3712001

Deloitte Global. (2025). Deloitte reports FY2025 revenue of $70.5 billion. Deloitte Press Room.

Deloitte. (2024). The new organizational knowledge management. Deloitte Insights.

Eesel AI. (2025). Notion AI limitations & best practices: A 2025 guide.

Glean. (2025). Glean raises $150M Series F at $7.2B valuation. Glean Press.

Hitt, M. A., Ireland, R. D., & Hoskisson, R. E. (2017). Strategic Management: Concepts and Cases (12th ed.). Cengage Learning.

Microsoft. (2023). Work Trend Index: Will AI Fix Work? https://www.microsoft.com/en-us/worklab/work-trend-index/will-ai-fix-work

Microsoft Learn. (2025). Microsoft 365 Copilot Retrieval API Overview.

Mordor Intelligence. (2026). Enterprise Search Market — 9%+ CAGR Forecast for 2026-2031. GlobeNewswire.

Progent. (2025). Programmable Privilege Control for LLM Agents. arXiv:2504.11703.

Question Base. (2026). Enterprise Search vs. Slack AI: Key Differences.

Service Performance Insight (SPI Research). (2025). Professional Services Maturity Benchmark Report.

Slite. (2025). Enterprise Search Survey Report. https://slite.com/en/learn/enterprise-search-survey-findings

Tian, F. et al. (2024). Customized FinGPT Search Agents Using Foundation Models. arXiv:2410.15284.

Yao, J., et al. (2024). A Survey on Large Language Model (LLM) Security and Privacy. arXiv:2312.02003.